II. Physikalisches Anfängerpraktikum

UNIVERSELLE HEIDELBEERE
EIGENTLICH EINE BROOMBEERE

Protokoll zum Versuch

Zähigkeit von Flüssigkeiten

(Versuch 212)

Autor: Finn Zeumer (hz334)
Versuchspatnerin: Annika Künstle
Versuchsbegleiter: Dimitrij Bathauer

Datum der Ausführung: 25.02.26
Abgabedatum: 8. März 2026

FEZ

Inhaltsverzeichnis

1. Einleitung **3**
1.1 Aufgabe und Motivation . . . 3
1.2 Physikalische Grundlage . . . 3
1.2.1 Laminare und turbulente Strömung . . . 4
1.2.2 Kugelfallviskosimeter nach Stoke . . . 4
1.2.3 Viskositätsbestimmung nach Hagen-Poiseuille . . . 4

2. Messdaten **6**

3. Durchführung **8**
3.1 Messverfahren . . . 8

4. Auswertung **11**
4.1 Bestimmung der Viskosität nach Stokes mit einem Kugelfallviskosimeter . . . 12
4.1.1 Gültigkeit des Gesetztes von Stokes . . . 14
4.2 Bestimmung der Viskosität nach Hagen-Poiseuille mit einem Kapillarviskosimeter . . . 16

5. Diskussion **19**
5.1 Zusammenfassung . . . 19
5.1.1 Stokes . . . 19
5.1.2 Hagen-Poiseuille . . . 19
5.2 Analyse der Messwerte . . . 20
5.3 Kritik . . . 21

6. Phython-Code **22**

1. Einleitung

Die Ausarbeitung der Einleitung ist anhand des Skriptes von Jens Wagner entstanden. Alle Abbildungen, sind entweder aus dem Skript oder selbst erzeugt, wenn nicht anders vermekrt. Rechtschreibprüfungen werden sowohl durhc K.I.-Modelle, sowie durch den Rechtschreibprüfer des Dudens durchgeführt. Dies gilt nur für die Einleitung und die Durchführung. Die Auswertung wird völlig ohne K.I. Assistenz geschrieben. [Wag26, Dud26]

1.1 Aufgabe und Motivation

Ziel des Versuchs ist die Bestimmung der dynamischen Viskosität η von Polyethylenglykol (PEG) mit zwei komplementären Messverfahren: dem Kugelfallviskosimeter nach Stokes und dem Kapillarviskosimeter nach Hagen-Poiseuille. Beim Kugelfallversuch wird zusätzlich der Übergang von laminarer zu turbulenter Umströmung der Kugel untersucht, um die Gültigkeitsgrenze des Stokes'schen Gesetzes zu bestimmen. Die mit beiden Methoden erhaltenen Werte werden abschließend verglichen und auf Konsistenz unter Berücksichtigung der Messunsicherheiten geprüft.

1.2 Physikalische Grundlage

Die innere Reibung einer Newtonschen Flüssigkeit lässt sich durch das Verhältnis der tangentialen Scherkraft F zur Fläche A und zum Geschwindigkeitsgradienten $\mathrm{d}v/\mathrm{d}z$ beschreiben. Diese Beziehung definiert die dynamische Viskosität η:

$$F = \eta A \frac{\mathrm{d}v}{\mathrm{d}z}. \tag{1.1}$$

Die Einheit von η ist $\mathrm{Pa} \cdot \mathrm{s}$.

Für eine Kugel mit Radius r, die sich mit konstanter Geschwindigkeit v durch eine (viskose) Flüssigkeit bewegt, gilt das Stokes'sche Gesetz für die Reibungskraft:

$$F_\mathrm{R} = 6\pi\,\eta\,r\,v. \tag{1.2}$$

Da in einem realen Fallrohr der Raum nicht unendlich ist, wird die Reibung mit einem Korrekturfaktor λ (Ladenburgsche Korrektur) angepasst; die Näherung lautet

$$\lambda = 1 + \alpha \frac{r}{R}, \tag{1.3}$$

wobei R der Radius des Fallrohres ist und α in diesem Versuch 2.1 beträgt. Die korrigierte Reibung ist dann die Multiplikation von Gleichung 1.2 mit Gleichung 1.3.

$$F_\mathrm{R} = 6\pi\eta r v\,\lambda \tag{1.4}$$

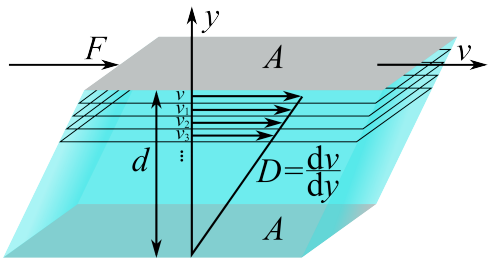


Abbildung 1.I: Gedankliche Darstellung der Viskosität. [Wik26]

Für eine fallende Kugel mit Materialdichte ρ_k, Flüssigkeitsdichte ρ_f und Kugelvolumen $V_k = \frac{4}{3}\pi r^3$ stellt sich im stationären Zustand ein Kräftegleichgewicht zwischen Gewichtskraft, Auftrieb und Reibung ein:

$$\underbrace{m\,g}_{F_g} - \underbrace{\rho_f V_k g}_{F_a} - \underbrace{6\pi\eta r\,\lambda\,v_s}_{F_r} = 0. \tag{1.5}$$

1.2.1 Laminare und turbulente Strömung

In der Physik unterscheidet man zwischen laminaren und turbulenten Strömungen. Dabei sind laminare Strömungen immer eine idealisierung. Rechnungen mit laminarer Strömung sind meist einfacher, aber immer als Nährung zu verstehen.

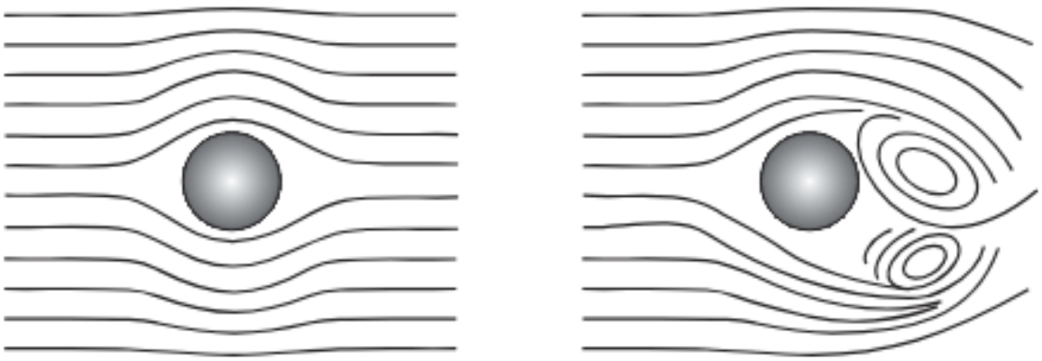


Abbildung 1.II:
Schematische Darstellung der Strömungsregime um eine Kugel: Laminar (links) mit symmetrischem Stromlinienmuster, (rechts) mit unregelmäßigen Wirbeln.

Um abzuschätze, wie lamianr eine Strömung ist, wird die Reynoldszahl eingeführt. Die dimensionslose Reynoldszahl charakterisiert das Verhältnis von Trägheits- zu Reibungskräften und ist definiert durch

$$\mathrm{Re} \;=\; \frac{\rho\, v\, L}{\eta}, \tag{1.6}$$

wobei L eine charakteristische Länge ist (für die Kugelbewegung $L = 2r$, für Rohrströmung $L = 2R$). Für Rohrströmungen liegt der Übergang zu Turbulenz typischerweise bei $\mathrm{Re} \approx 2300$; für die Umströmung einer Kugel ist der kritische Bereich deutlich kleiner (experimentell $\mathrm{Re}_{\mathrm{krit}} \approx 1$).

1.2.2 Kugelfallviskosimeter nach Stoke

Nachdem das in Gleichung 1.5 beschriebene Gleichgewicht eingesetzt hat, kann man diese Gleichung umstellen und nach η umstellen. Dabei kommt man auf das Ergebnis:

$$\eta = \frac{2}{9}\, g\, \frac{\rho_k - \rho_f}{v}\, r^2\,. \tag{1.7}$$

Somit ist es möglich die Viskosität aus der (gemessenen) Fallgeschwindigkeit zu bestimmen.-

1.2.3 Viskositätsbestimmung nach Hagen-Poiseuille

Zu Viskositätsbestimmung nach Hagen-Poiseuille wird nun die Kapillare am unteren Zylinderende betrachtet. Denn durch die Druckdifferenz $\Delta p_{\mathrm{diff}} = p_1 - p_2$ und die daduch entstehende Kraft $F_p = \pi r^2\, \Delta p_{\mathrm{diff}}$ fließt die Flüssigkeit aus der Kapillare. Dieser Kraft wirkt die Reibungskraft $F_R = 2\pi r\, L\, \eta\, \mathrm{d}v/\mathrm{d}r$.

Der Fließgeschwindigketi durch ein zylindrisches Rohr hat ein parabolisches Geschwindigketisprofil. Wir nehmen an, das am Zylinderrand die Fließgeschwindigkeit gleich null ist. Zudem wird die Annahme getroffen, dass es sich um eine laminare Strömung handelt.

Es wird sich wieder ein Gleichgewichtszustand zwischen F_R und F_p einstellen. Unter allen getrofenen Annahmen, lässt sich der Volumensstrom zu

$$\frac{\mathrm{d}V}{\mathrm{d}t} = \frac{\pi\, \Delta p_{\mathrm{diff}}\, R^4}{8\eta\, L} \tag{1.8}$$

herleiten. In dieser Gleichung ist R der Rohrradius und L die Rohrlänge. Durch Umstellen von Gleichung 1.8 nach η kommt man auf

$$\eta = \frac{\pi \, \Delta p_{\text{diff}} \, R^4}{8 L \, \dot{V}} \tag{1.9}$$

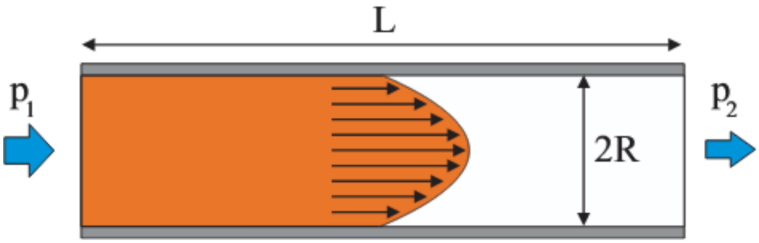


Abbildung 1.III:
Schematische Darstellung für den zweiten Versuchsteil: Eine Flüssigkeit fließt durch ein Rohr, wobei die Geschwindigkeitsverteilung parabolisch ist.

Amiha Künstle
Finn Zeuner
25.02.2026

Messprotokoll Versuch 212

Es soll die Viskosität η von PEG bestimmt werden. Hierz sollen die Methoden von Stokes und Hagen-Poiseville verwendet und im Nachhinein verglichen. Die Berücksichtigung äußerer Faktoren, sowie die Strömungsart sind besonders wichtig.

Aufgabe 1)

Bestimmung der Viskosität η nach Stokes.

Temperatur des PEG ist zu bestimmen.

Durchmesser des Kugelgefäßes: $d_K = 75$ mm

Raumtemperatur: $22{,}4° \pm 0{,}5°C$

Tabelle I) Sinkgeschwindigkeit nach Kugeldurchmesser

Kugeldurchmesser [mm]		Sinkstrecke [mm]	t_1 [s]	t_2 [s]	t_3 [s]	t_4 [s]	t_5 [s]	T [°C]	v [10^4 m/s]
9.000	± 0.025	200 ± 2.0	6,4 ±0,3	6,7	6,5	6,5	6,3	21.5 ±0.5	0.63 ± 0.03
8.000	± 0.025	200	7,6	7,8	8,7	7,7	8,3	21.5	2.00 ± 0.08
7.000	± 0.025	200	8,8	8,6	8,6	8,6	10,0	21.5	4.55 ± 0.07
6.000	± 0.025	200	13,9	13,3	12,9	13,0	13,5	21.5	7.48 ± 0.07
5.000	± 0.025	150	12,8	13,2	12,5	13,7	12,3	21.5	11.628 ± 0.126
4.000	± 0.025	150	20,6	20,0	20,0	20,2	19,5	21.5	15.018 ± 0.203
3.000	± 0.025	100	22,6	22,2	20,7	21,1	22,6	21.5	20.53 ± 0.18
2.000	± 0.025	50	26,0	23,7	26,1	24,7	24,4	21.5	25.3 ± 0.4
1.000	± 0.025	50	87,3	78,2	74,8	81,2	79,7	21.5	30.9 ± 0.3

Die Tabelle I zeigt die gegebenen Kugeldurchmesser, die Sinkstrecke Δs, die Temperatur T des PEG und die Sinkzeit der i-ten Kugel t_i.

Die Geschwindigkeit $v = \frac{\Delta s}{t}$.

Aufgabe 2)

Es soll die Viskosität von PEG nach Hagen-Poiseuille bestimmt werden.

$h_A =$ (537 ± 3) mm , $h_E =$ (524 ± 3) mm

Raumtemperatur: (22.4 ± 0.5) °C , PEG-Temp.: (27.5 ± 0.5) °C

Tabelle III) Volumenfluss des PEG

V [cm³]	t [s]	Differenz zum Vorwert Δt
5.00 ± 0.10	92.4 ± 1.0	
10.00	233.8	
15.00	382.8	
20.00	540.3	
25.00	688.4	
30.00	847.3	

Der Fehler für das Ablesen ist der doppelte Ablesefehler, da die Oberfläche gekrümmt ist.

Tabelle III zeigt das Füllvolumen V des Messkolbens und die Zeit t wann V erreicht wurde.

Badhree

3. Durchführung

Auch der Folgende Text ist am Praktikumsskript von Jens Wagner orientiert. Es gelten diesleben Hinweise wie in Kapitel 1..[Wag26, Dud26]

3.1 Messverfahren

Stokes Im ersten Versuchsteil werden POM-Kugeln verschiedener Durchmesser möglichst mittig in den Fallzylinder fallengelassen, welcher mit PEG Gefüllt ist. Es wird sich für jede Kugel eine Fallstrecke heruasgesucht, welche an der Skala abgelesen werden kann. Für diese Strecke wird die Sinkzeit der jeweiligen Kugel per Stoppuhr gemessen. Größere Kugeln sinken scheller, daher wird hier eine größere Strecke gewählt, um die Messfehler kleinzuhalten. Jeder Kugeldurchmesser wird fünf mal gemessen und final wird die durchschnittliche Sinkgeschwindigkeit bestimmt. Es gibt neun Durchmesser, von 1mm bis 9mm. Die jeweilige Fallzeit ist dem Messprotokoll zu entnehmen.

Hagen-Poiseuille Für diesen Versuchsteil wird nun die Kapillare am unteren Zylinderende betrachtet. Wird das Ventil geöffnet, so ströt das PEG durch die Kapillare in einen Messzylinder. Nachdem sich nach dem Öffnen ein konstanter Massenfluss eingestellt hat, wird eine Zeitstoppung gestartet. Immer wenn sich das Volumen um 5cm^3 verändert hat, wird die aktuelle Zeit aufgeschrieben. Somit lässt sich der Volumenfluss bestimmen.

Skizze des Versuchsaufbaus

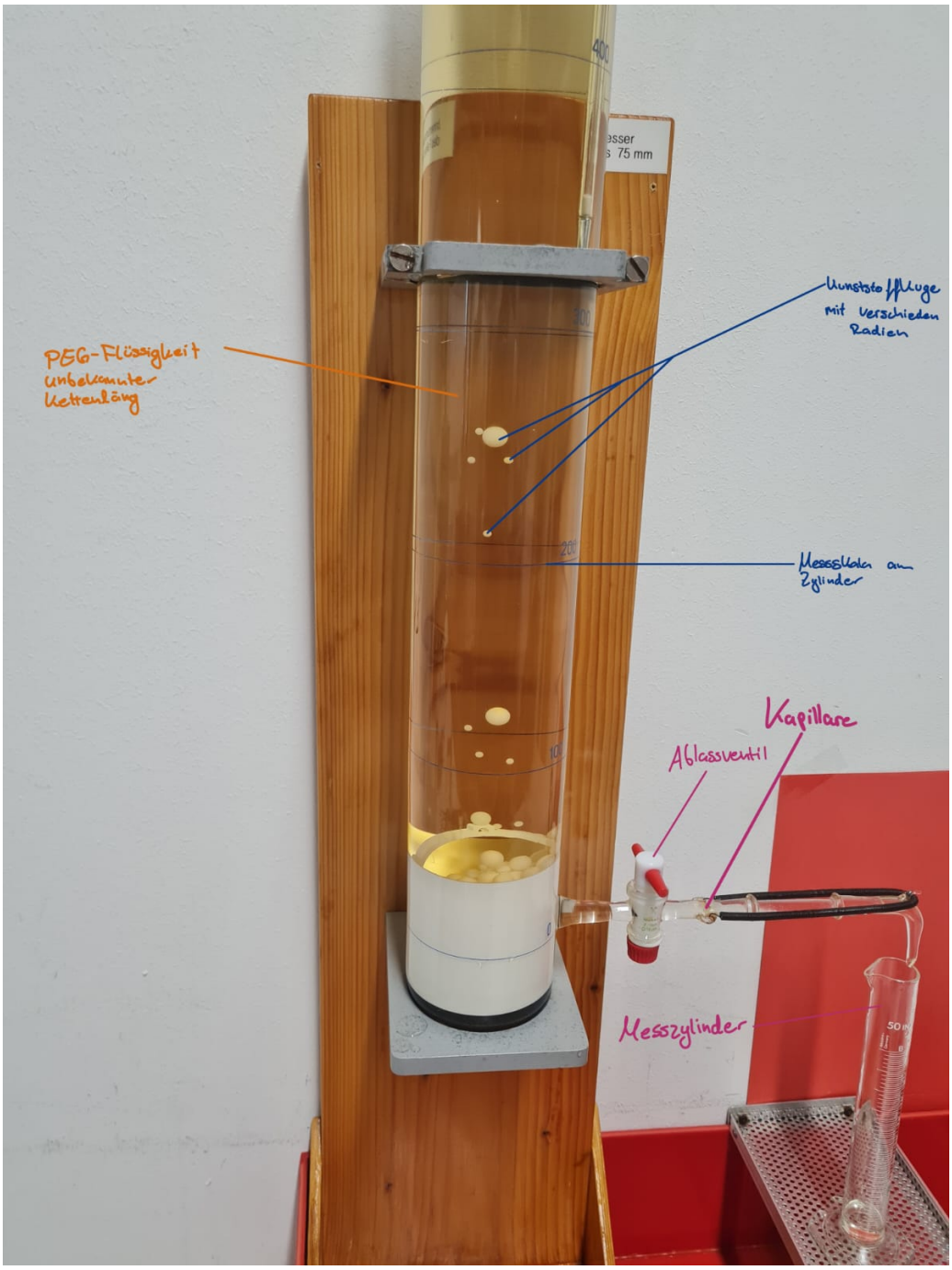


Abbildung 3.I:
Dartsellung des Versuchaufbaus. Orange: Wichtig für beide Aufgabenteile. Blau: Wichtig zur Bestimmung nach Stokes. Magenta: Wichtig zur Bestimmung nach Hagen-Poiseuille

Abbildung 3.II:
Dartsellung des Versuchaufbaus. Orange: Wichtig für beide Aufgabenteile. Blau: Wichtig zur Bestimmung nach Stokes. Magenta: Wichtig zur Bestimmung nach Hagen-Poiseuille

4. Auswertung

Fehlerrechnung

Für die statistische Auswertung von n Messwerten x_i werden folgende Größen definiert [Wag25]:

$$\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i \qquad \text{Arithmetisches Mittel} \tag{4.1}$$

$$\sigma^2 = \frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})^2 \qquad \text{Variation} \tag{4.2}$$

$$\sigma = \sqrt{\frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})^2} \qquad \text{Standardabweichung} \tag{4.3}$$

$$\Delta\bar{x} = \frac{\sigma}{\sqrt{n}} = \sqrt{\frac{1}{n(n-1)}\sum_{i=1}^{n}(\bar{x} - x_i)^2} \qquad \text{Fehler des Mittelwerts} \tag{4.4}$$

$$\Delta f = \sqrt{\left(\frac{\partial f}{\partial x}\Delta x\right)^2 + \left(\frac{\partial f}{\partial y}\Delta y\right)^2} \qquad \text{Gauß'sches Fehlerfortpflanzungsgesetz für } f(x,y) \tag{4.5}$$

$$\Delta f = \sqrt{(\Delta x)^2 + (\Delta y)^2} \qquad \text{Fehler für } f = x + y \tag{4.6}$$

$$\Delta f = |a|\Delta x \qquad \text{Fehler für } f = ax \tag{4.7}$$

$$\frac{\Delta f}{|f|} = \sqrt{\left(\frac{\Delta x}{x}\right)^2 + \left(\frac{\Delta y}{y}\right)^2} \qquad \text{relativer Fehler für } f = xy \text{ oder } f = x/y \tag{4.8}$$

$$\sigma = \frac{|a_{lit} - a_{gem}|}{\sqrt{\Delta a_{lit}^2 + \Delta a_{gem}^2}} \qquad \text{Berechnung der signifikanten Abweichung} \tag{4.9}$$

4.1 Bestimmung der Viskosität nach Stokes mit einem Kugelfallviskosimeter

Es soll zunächst die Viskosität η von Polyethylenglykol (PEG) mit einem Kugelfallviskosimeter nach Stokes bestimmt werden. Dazu werden die Fallzeiten t von Kugeln mit verschiedenen Durchmessern d über eine definierte Strecke s gemessen, um die mittlere Sinkgeschwindigkeit $v = s/\bar{t}$ zu berechnen. Die ladenburgsche Korrektur λ wird angewendet, um die Endgeschwindigkeit v_s zu korrigieren, und die Viskosität η wird aus der umgestellten Stokes'schen Formel (Gleichung 1.7) bestimmt. Es wird auch der Übergang von laminarer zu turbulenter Umströmung untersucht. Zunächst wird die mittlere Sinkzeit über das arithmetische Mittel (4.1) bestimmt:

$$\bar{t} = \frac{1}{5}\sum_{i=1}^{5} t_i \tag{4.10}$$

Die Ungenauigkeit wird via Gauß'scher Fehlerfortpflanzung (4.5) bestimmt, dabei wird die Standardabweichung der Messwerte über die Standardabweichung (4.3) berechnet und durch die Wurzel der Anzahl der Messwerte geteilt, um den Fehler des Mittelwerts zu erhalten:

$$\Delta\bar{t} = \frac{\sqrt{(\Delta t)^2 + \sigma^2}}{\sqrt{5}} \tag{4.11}$$

Dabei ist Δt die Ungenauigkeit der Zeitmessung, welche durch die Reaktionszeit des Experimentators bestimmt wird, wobei die Genauigkeit der Stoppuhr dagegen vernachlässigt werden kann. Aus der gemessenen Fallzeit t wird die mittlere Sinkgeschwindigkeit v über die Formel $v = s/\bar{t}$ berechnet, wobei s die definierte Strecke ist. Die Ungenauigkeit der Geschwindigkeit Δv wird ebenfalls über die Gauß'sche Fehlerfortpflanzung bestimmt, wobei die Unsicherheiten von s und $\bar{t}$ berücksichtigt werden:

$$\Delta v = \sqrt{\left(\frac{1}{\bar{t}}\Delta s\right)^2 + \left(\frac{s\Delta\bar{t}}{\bar{t}^2}\right)^2}. \tag{4.12}$$

Damit sind die wichtigen Parameter zur Berechnung der Viskosität η über die umgestellte Stokes'sche Formel gegeben. Über die Gleichung 1.7 kann die Viskosität berechnet werden. Dazu wird zunächst die Sinkgeschwindigkeit v gegen den Quadratradius der Kugeln geplottet. Der Quadratradius ist definiert als:

$$r^2 = \left(\frac{d}{2}\right)^2. \tag{4.13}$$

Seine Ungenauigkeit wird über die Gauß'scher Fehlerfortpflanzung (4.5) bestimmt, wobei die Unsicherheit des Durchmessers d berücksichtigt wird:

$$\Delta r^2 = \frac{d}{2}\Delta d. \tag{4.14}$$

Die Ungenauigkeiten für die Messzeiten ($\Delta t = 0.3$s) und die Durchmesser ($\Delta d = 0.0025$mm) sind dem Messprotokoll csecmessprotokoll zu entnehmen. Die Unsicherheit der Strecke ($s = \{0.002$m) wird ebenfalls berücksichtigt, wobei sie durch die Genauigkeit der Skala geschätzt wird.

Darüber hinaus muss noch die ladenburgsche Korrektur λ (Gleichung 1.3) berechnet werden. Seine Unsicherheit $\Delta\lambda$ wird ebenfalls über die Gauß'sche Fehlerfortpflanzung bestimmt, wobei die Unsicherheiten der Durchmesser d und des Fallrohrs D [1] berücksichtigt werden, welcher in diesem Versuch als exakt angenommen wird:

$$\Delta\lambda = \frac{2,1}{D}\Delta d \tag{4.15}$$

[1] Urspünglich d_k genannt, für bessere Übersicht in D unbenannt.

Damit lässt sich die tatsächliche Endgeschwindigkeit v_s über die Formel $v_s = v \cdot \lambda$ berechnen, wobei die Unsicherheit Δv_s ebenfalls über die Gauß'sche Fehlerfortpflanzung bestimmt wird, wobei die Unsicherheiten von v und λ berücksichtigt werden:

$$\Delta v_s = \sqrt{(\lambda \Delta v)^2 + (v \Delta \lambda)^2}. \tag{4.16}$$

Aus den in Tabelle 4.I zusammengefassten Messwerten können nun die Werte für den Plot von v gegen r^2 berechnet werden.

Tabelle 4.I:
Wichtigen Messwerte aus dem Messprotokoll zusammengefasst.

d $[\mathrm{m} \cdot 10^{-6}]$	r $[\mathrm{m} \cdot 10^{-6}]$	s $[\mathrm{m} \cdot 10^{-6}]$	$\bar{t}$ [s]
100.0 ± 2.5	50.0 ± 1.3	5.00 ± 0.20	79.04 ± 1.24
200.0 ± 2.5	100.0 ± 1.3	5.00 ± 0.20	25.0 ± 0.6
300.0 ± 2.5	150.0 ± 1.3	10.00 ± 0.20	22.0 ± 0.5
400.0 ± 2.5	200.0 ± 1.3	15.00 ± 0.20	20.1 ± 0.3
500.0 ± 2.5	250.0 ± 1.3	15.00 ± 0.20	12.9 ± 0.4
600.0 ± 2.5	300.0 ± 1.3	20.00 ± 0.20	13.3 ± 0.3
700.0 ± 2.5	350.0 ± 1.3	20.00 ± 0.20	9.7 ± 0.3
800.0 ± 2.5	400.0 ± 1.3	20.00 ± 0.20	7.9 ± 0.3
900.0 ± 2.5	450.0 ± 1.3	20.00 ± 0.20	6.5 ± 0.3

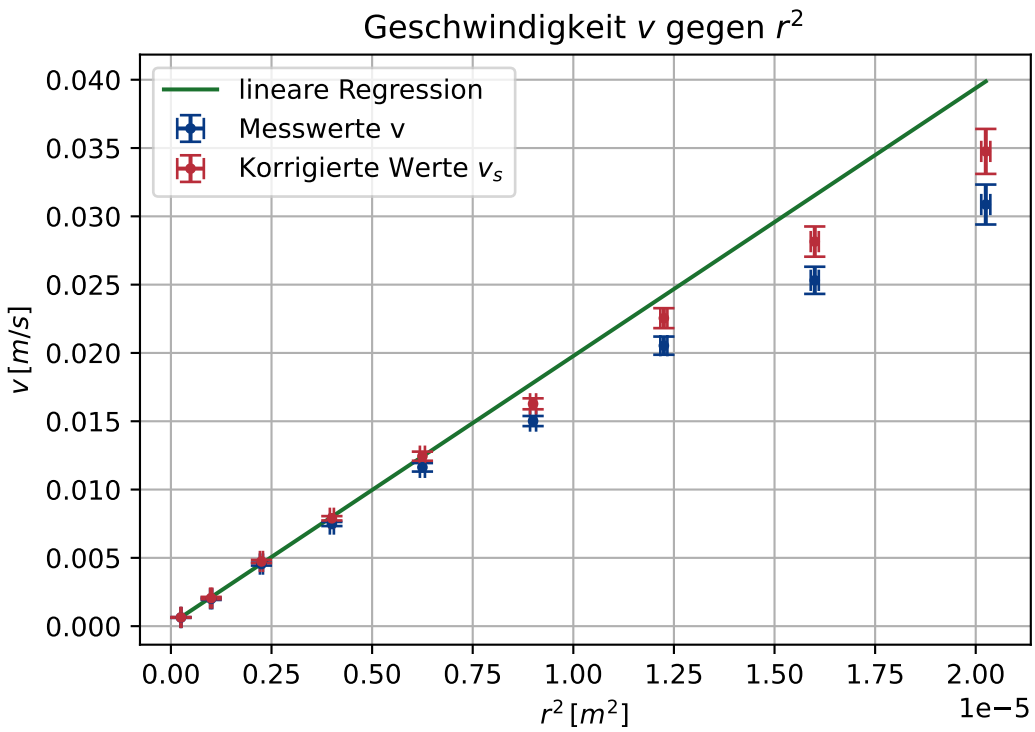


Abbildung 4.I:
Plot der Geschwindigkeiten gegen den quadrierten Kugelradius. Die Lineare Regression ist durch die ersten fünf Messwerte der korrigierten Werte gelegt. Die lineare Nährung zeigt sich damit als nicht gültig.

Es lässt sich beobachten, dass für kleine Kugelradien die gemessenen Werte v und die korrigierten

Werte v_s nahezu identisch sind, was darauf hindeutet, dass die ladenburgsche Korrektur in diesem Bereich vernachlässigbar ist. Für größere Kugelradien hingegen weichen die unkorrierten Werte v deutlich von den korrigierten Werten v_s ab, was die zunehmende Bedeutung der ladenburgschen Korrektur bei größeren Kugeln verdeutlicht.

Jedoch ist auch zu beobachten, dass selbst die korrigierten Werte v_s für die größten Kugeln nicht mehr der linearen Beziehung entsprechen, die durch die Stokes'sche Formel (Gleichung 1.7) beschrieben wird. Dies deutet darauf hin, dass bei größeren Kugeln der Übergang von laminarer zu turbulenter Umströmung stattfindet, wodurch die Stokes'sche Formel nicht mehr gültig ist.

Die Steigung ist via $k = \Delta v_s / \Delta r^2$ definiert. Seine Ungenauigkeit ist somit

$$ErdbeschleunigunginHeidelbe\Delta k = \sqrt{\left(\frac{\Delta v_s}{r^2}\right)^2 + \left(\frac{2\, v_s\, \Delta r}{r^3}\right)^2}\,. \tag{4.17}$$

Final berechnet sich die Steigung zu

$$\underline{k = (1961 \pm 25)\,\frac{1}{\mathrm{m\,s}}} \tag{4.18}$$

Somit kann die Viskosität η über die umgestellte Stokes'sche Formel Gleichung 1.7 berechnet werden, und auch hier muss die Unsicherheit $\Delta\eta$ ebenfalls über die Gauß'sche Fehlerfortpflanzung (4.5) bestimmt wird, wobei die Unsicherheiten der Steigung k und der Dichteunterschied $\rho_k - \rho_f$ berücksichtigt werden:

$$\Delta\eta = \eta \sqrt{\frac{\Delta pf^2}{(p_f - p_k)^2} + \frac{\Delta pk^2}{(p_f - p_k)^2} + \frac{\Delta k^2}{k^2} + \frac{\Delta g^2}{g^2}}\,. \tag{4.19}$$

Dabei ist g die Ortskonstante der Erdbeschleunigung in Heidelberg mit $g = (9.80984 \pm 0.00002)\,\mathrm{m/s^2}$, $\rho_k = (1385 \pm 15)\,\frac{\mathrm{kg}}{\mathrm{m^3}}$ die Dichte der POM-Kugeln nach Angaben im Praktikumsskript. Die Steigung k ist gemäß Gleichung 4.1 angegeben. Die Dichte des PEG ist in Abhänigkeit der Temperatur zu bestimmen. Dem Messprotokoll ist zu entnehmen, dass die PEG-Temperatur konstant den Wert $T_1 = (21.5 \pm 0.5)°\mathrm{C}$ gehalten hat. Um zu überprüfen, ob der Fehler Unterschätzt wurde, soll noch die Temperatur $T_2 = (21.5 \pm 1.0)°\mathrm{C}$ untersucht werden. Die Werte lassen sich ungerundet der Abbildung 4.II entnehmen.

Für T_1 kommt man dabei auf eine Dichte von

$$\underline{\rho_{f,1} = (1148.2 \pm 0.5)\,\frac{\mathrm{kg}}{\mathrm{m^3}}} \tag{4.20}$$

und für T_2 auf

$$\underline{\rho_{f,2} = (1148.2 \pm 1.2)\,\frac{\mathrm{kg}}{\mathrm{m^3}}} \tag{4.21}$$

Interessant ist, egal welchen Fehler man hier einsetzt, das Ergebnis läuft auf

$$\boxed{\eta_\mathrm{s} = (0.263 \pm 0.017)\,\frac{\mathrm{kg}}{\mathrm{m\,s}}} \tag{4.22}$$

hinaus.

Das liegt daran, dass der Fehler sowieso vernachlässigbar gegenüber der anderen Fehlergrößen ist. Es soll daher mit T_1 weiter gerechnet werden.

4.1.1 Gültigkeit des Gesetztes von Stokes

Es soll nun untersucht werden, wie groß der Gültigkeitsbereich des Gesetzes von Stokes ist. Dafür wird das Ergebnis aus Sektion 4.1 zur Hilfe genommen. Zur Prüfung wird die theoretisch erwatete Sinkgeschwindigkeit v_lam nach Gleichung 4.23 benutzt und der Reynoldsfaktor Re wird bestimmt.

$$v_\mathrm{lam} = \frac{2}{9} g\, \frac{(p_k - p_f)}{\eta\, \lambda}\, r^2 \tag{4.23}$$

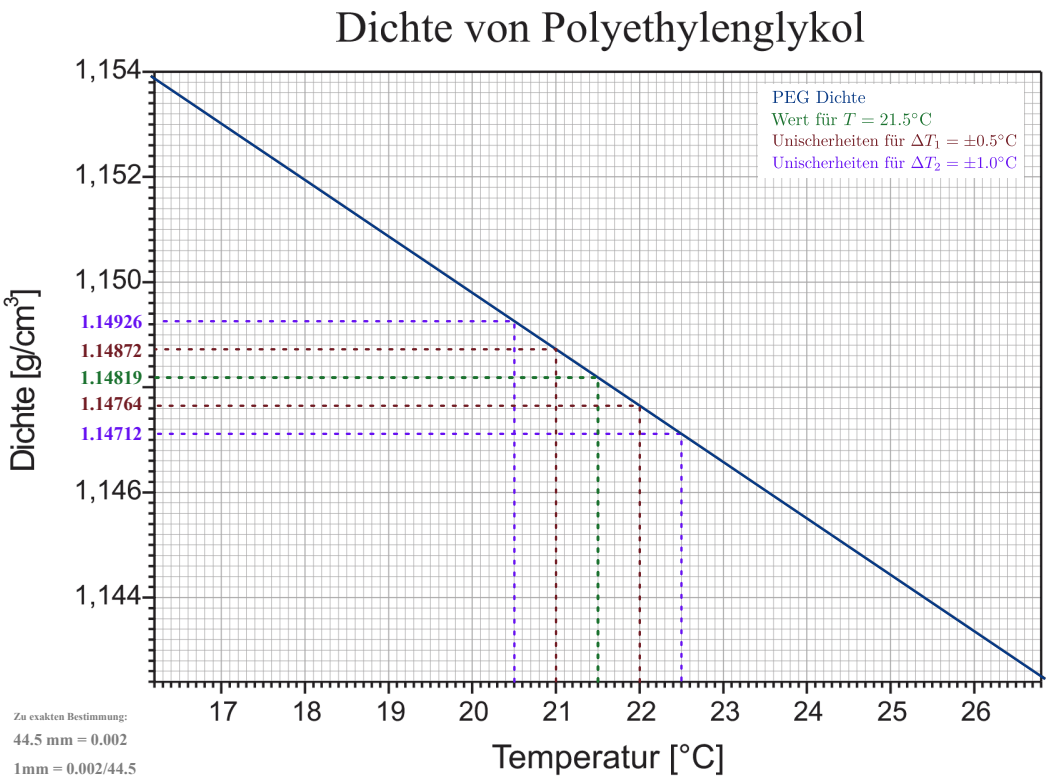


Abbildung 4.II:
Graph der Dichte von PEG in Temperaturabhänigkeit. Eingezeichnet sind die Werte für die Temperatur $T_1 = (21.1 \pm 0.5)°C$ und die für $T_2 = (21.5 \pm 1.0)°C$. Die Werte sind noch nicht gerundet. Bestimmt wurden die Werte über das Grafikprogramm „Inkscape“.

$$\Delta v_{\text{lam}} = v_{\text{lam}} \sqrt{\frac{\Delta p_f^2}{(p_f - p_k)^2} + \frac{\Delta p_k^2}{(p_f - p_k)^2} + \frac{4\Delta r^2}{r^2} + \frac{\Delta v_{\text{lam}}^2}{v_{\text{lam}}^2} + \frac{\Delta g^2}{g^2} + \frac{\Delta \eta^2}{\eta^2}} \tag{4.24}$$

Die Berechnung der Reynoldszahl ist in Gleichung 1.6 festgehalten. Der Fehler berechnet sich zu

$$\Delta \text{Re} = \text{Re} \sqrt{\frac{\Delta v^2}{v^2} + \frac{\Delta \rho_f^2}{\rho_f^2} + \frac{\Delta \eta^2}{\eta^2} + \frac{\Delta L^2}{L^2}}\,. \tag{4.25}$$

Dabei ist $L = 2r$. DIe Ergebnisse der Berechnung sind der Tabelle 4.II zu entnehmen.

Um die kritische Reynolds-Zahl zu bestimmen, wird das Verhältnis der korrigierten Geschwinigkeit v_s und der theoretischen Geschwindigkeit v_{lam} gegen $lg(Re)$ geplottet. In der Abbildung 4.III ist leider kein „Knick“ zu erkenne. Normalerweise were dies der Wert, ab dem man die kritische Reynoldszahl abliest. Grob vermuten lässt sich jedoch, dass es der siebte Wert, in der Abbildung 4.III rot markiert, ist.

Die Vermutung, dass es rund um den siebten Wert liegt, stammt auch daher, dass bekannt ist, dass ein Wert von < 1 erwartet wird. Die zwei Folge werte scheinen auch dem linearen Verlauf zu folgen, der erwatet werden würde, der Wert davor jedoch nicht. Um die Unsicherheit aber klar zu machen, wird ein großzügiger Fehler von 50% angenommen. Somit kommt man auf einen kritischen Wert von:

$$\boxed{Re_{\text{krit}} = (0.6 \pm 0.3)} \tag{4.26}$$

Tabelle 4.II:
Theoretische Sinkgeschwindigkeit v_{lam} und Reynoldszahl in Abhänigkeit von r, mit der Ungenauigkeit $\Delta r = 0.013\text{m} \cdot 10^{-3}$.

$r[\text{m} \cdot 10^{-3}]$	$v_{\text{lam}} \frac{\text{m}}{\text{s}}$	Re
0.5	0.48 ± 0.05	0.00276 ± 0.00022
1.0	1.91 ± 0.18	0.0175 ± 0.0014
1.5	4.2 ± 0.4	0.060 ± 0.004
2.0	7.4 ± 0.7	0.131 ± 0.009
2.5	11.5 ± 1.0	0.254 ± 0.018
3.0	16.3 ± 1.5	0.393 ± 0.027
3.5	21.90 ± 1.10	0.63 ± 0.05
4.0	28.2 ± 2.6	0.88 ± 0.07
4.5	35 ± 3	1.123 ± 0.010

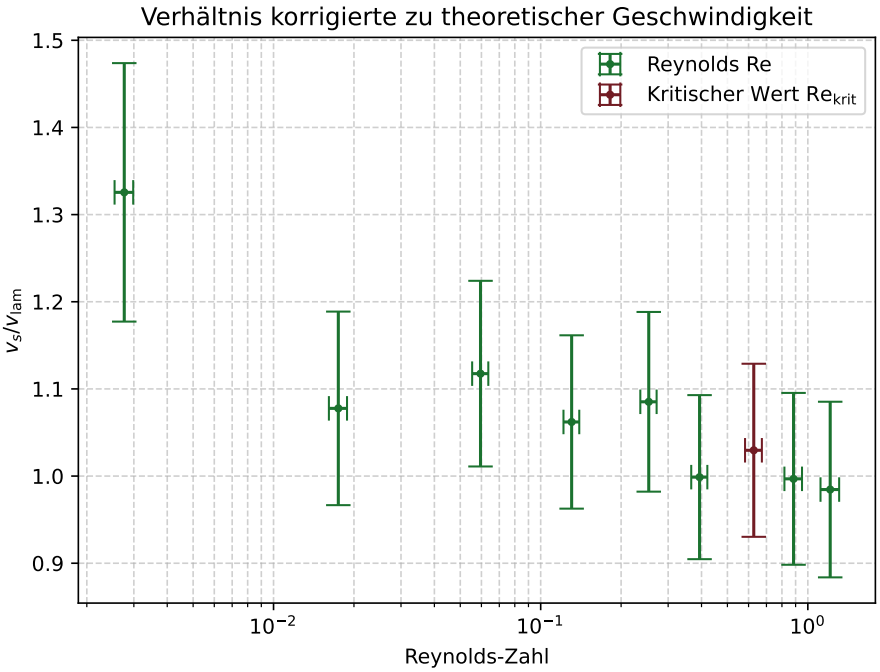


Abbildung 4.III:
Keine eindeutigen Messergebnisse. Der kritische Punkt wird nur erahnt.

4.2 Bestimmung der Viskosität nach Hagen-Poiseuille mit einem Kapillarviskosimeter

Nun soll sich im letzten Aufgabenteil einer neuen Methode zur Bestimmung der Viskosität Augenmerk geschenckt werden. Bei der Hagen-Poiseuille-Methode wird der Volumenfluss nach Gleichung 1.8 bestimmt. Die Messwerte sind dem Messprotokoll der Tabelle II zu entnehmen. Der Rohrdurchmesser ist mit $R = (1.50 \pm 0.01)$mm und seine Länge mit $L = (100.0 \pm 0.5)$mm gegeben. [Wag26]

In Abbildung 4.IV sind die Messwert inklusive linerarer Regression eingezeichnet. Die Steigung der Geraden entspricht dabei gerade dem Volumenfluss:

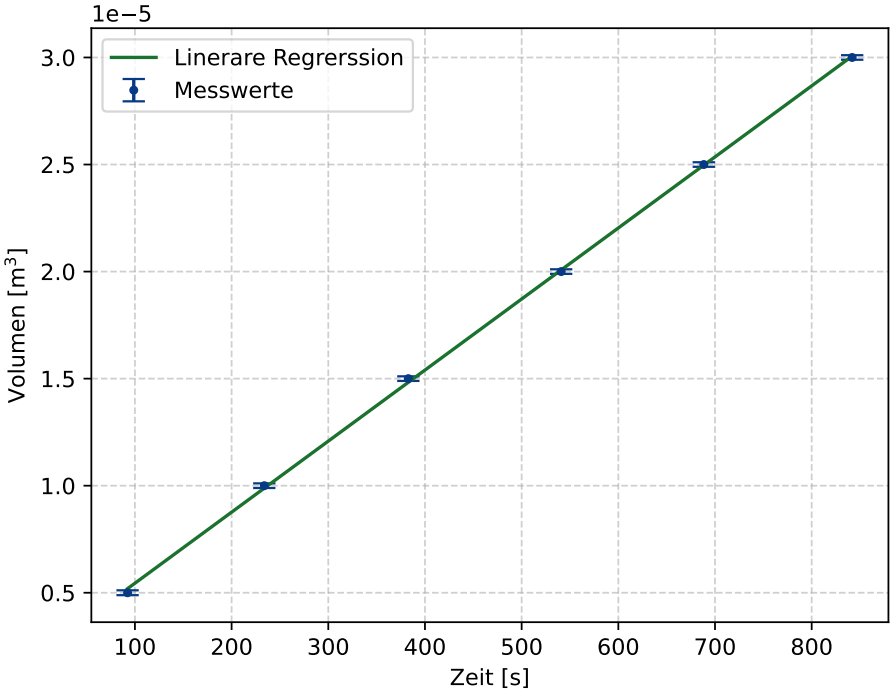


Abbildung 4.IV:
Plot der Messwerte (blau) mit linerarer Regression (grün).

$$\underline{\dot{V} = (3.318 \pm 0.017) \cdot 10^{-8}\,\frac{\mathrm{m}^3}{\mathrm{s}}} \tag{4.27}$$

Weiterführend fehlt nun noch die Druckdifferenz $p_{\mathrm{diff}} = p_1 - p_2$. Dabei entspricht p_2 dem Athmosphärendruch und p_1 dem Druck, der durch die Schwerkraft auf das PEG wirkt(„hydrostatischer Druck“). Dieser ist nur Abhänig von der Füllhöhe des Zylinders. Somit lässt sich die Druckdifferenz auch wie folgt formulieren:

$$p_{\mathrm{diff}} = \frac{\rho_f\, g}{2}\,(h_{\mathrm{A}} - h_{\mathrm{E}})\,, \tag{4.28}$$

und seine Ungenauigkeit

$$\Delta p_{\mathrm{diff}} = p_{\mathrm{diff}} \sqrt{\frac{\Delta h_{\mathrm{A}}^2}{(h_{\mathrm{A}} + h_{\mathrm{E}})^2} + \frac{\Delta h_{\mathrm{E}}^2}{(h_{\mathrm{A}} + h_{\mathrm{E}})^2} + \frac{\Delta g^2}{g^2} + \frac{\Delta \rho_f^2}{\rho_f^2}}\,. \tag{4.29}$$

Berechnet man mit den bekannten Werten die Druckdifferent, so kommt man auf

$$\underline{p_{\mathrm{diff}} = (5941 \pm 24)\,\mathrm{Pa}} \tag{4.30}$$

Damit sind alle Werte zusammen, die für die Berechnung der Viskosität nach Hagen-Poiseuille benötigt werden. Die Gauß'sche Fehlerfortpflanzung (4.5) wird auf Gleichung 1.9 angewendet:

$$\Delta\eta = \eta \sqrt{\frac{\Delta p^2}{p^2} + \frac{\Delta \dot{V}^2}{\dot{V}^2} + \frac{16 \Delta R^2}{R^2} + \frac{\Delta L^2}{L^2}} \tag{4.31}$$

Somtit kommt man auf eine Viskosität von

$$\boxed{\eta_{\mathrm{hp}} = (0.222 \pm 0.006)\,\frac{\mathrm{kg}}{\mathrm{m\,s}}} \tag{4.32}$$

nach der Methode von Hagen-Poiseuille.

Reynoldszahl für Hagen-Poiseuille

Zu letzt soll noch die Reynolds-Zahl bestimmt werden. Dafür benötigen wir die mittlere Fließgeschwindigkeit $\overline{v}_f$. Diese Berechnet sich nach

$$\overline{v}_f = \frac{\dot{V}}{\pi\,R^2}\,, \tag{4.33}$$

welche im Zusammenhang mit der Reynoldszahl steht:

$$Re_{\mathrm{hp}} = \frac{\rho_f\,\overline{v}_f\,2r_{\mathrm{k}}}{\eta} \tag{4.34}$$

Durch Einsetzen von Gleichung 1.9 und Gleichung 4.33 in Gleichung 4.34 erhält man

$$Re_{\mathrm{hp}} = \frac{2\dot{V}\rho_f}{\pi L R} \tag{4.35}$$

und seine Ungenauigkeit nach Gauß'scher Fehlerfortpflanzung (4.5)

$$\Delta Re_{\mathrm{hp}} = Re_{\mathrm{hp}}\sqrt{\frac{\Delta\rho_f^2}{\rho_f^2} + \frac{\Delta\dot{V}^2}{\dot{V}^2} + \frac{\Delta R^2}{R^2} + \frac{\Delta L^2}{L^2}}\,. \tag{4.36}$$

Durh einsetzen der Messwerte kommt man final auf eine Reynodls-Zahl von

$$\boxed{Re_{\mathrm{hp}} = (0.1617 \pm 0.0016)} \tag{4.37}$$

Für die Hagen-Poiseuille-Methode gilt für Reynoldszahlen ab $Re \approx 2300$, dass die Strömng turbulent wird. Der bestimmte Wert liegt dabei weit unter den kritischen Wert.

5. Diskussion

5.1 Zusammenfassung

Das Ziel des Versuches war es, die Viskosität von PEG zu bestimmen, sowie zu schauen, ob die jeweilige Nährung von laminarer Strömung Gültigkeit hat. Die Bedetung der Messergebnisse soll in Sektion 5.2 diskutiert werden.

5.1.1 Stokes

In diesem Versuchsteil sollte, wie erwähnt, die Viskosität des PEG bestimmt werden, die theoretische Sinkgeschwindigkeit bestimmt werden und geschaut werden, für welche Kugelradien die Nährung für eine laminare Strömung gültig ist.

Viskosität Zur Bestimmung der Viskosität wurde die mittlere Sinkgeschwindigkeit der einzelnen Kurgeln bestimmt. Zudem musste die Dichte des PEG anhand seiner Temperatur bestimmt werden. Somit waren alle Werte zur Bestimmung zusammen.

Über die Messergebnisse konnte die Viskosität von

$$\boxed{\eta_\mathrm{s} = (0.263 \pm 0.017)\,\frac{\mathrm{kg}}{\mathrm{m\,s}}} \tag{5.1}$$

bestimmt werden.

theoretische Sinkgeschwindigkeit Die Messergebnisse der Sinkgeschwindigkeit wurden mit der Ladenburg-Korrektur nachverbessert. Später wurde dann die theoretisch Sinkgeschwindigkeit in laminarer Strömung bestimmt. Die Ergebnisse sind tabellarisch im Messprotokoll und Tabelle 4.II festgehalten.

Reynoldszahl Für die Reynodls-Zahl wurde der Plot in Abbildung 4.III erstellt. Dieser hatte nicht die erhoffte Form. Der kritische Wert musste daher unter Annahme eines großen Fehlers abgelesen werden:

$$\boxed{Re_\mathrm{krit} = (0.6 \pm 0.3)} \tag{5.2}$$

Dabei wurde festgestellt, das die Ungenauigkeit der Temperatur keine besonders wichtige Rolle spielt und der Wert von $\Delta T = \pm 0.5\,°\mathrm{C}$ angenommen werden kann.

5.1.2 Hagen-Poiseuille

Auch in diesem Versuchsteil sollte die Viskosität des PEG bestimmt werden, sowie die Gültigkeit der laminaren Nährung geprüft werden.

Viskosität Die Viskosität nach Hagen-Poiseuille wurde über den Volumenfluss einer Kapillare bekannter Maße bestimmt. Dafür wurden die Messwerte geplottet und die Steigung bestimmt. Zuletzt wurde der Druck über die Höhendifferenz vor und nach der Zeitstoppung bestimmt. Aus allen Werten konnte auf die Viskosität

$$\boxed{\eta_\mathrm{hp} = (0.222 \pm 0.006)\,\frac{\mathrm{kg}}{\mathrm{m\,s}}} \tag{5.3}$$

geschlossen werden.

Reynoldszahl Über die durchschnittliche Ausflussgeschwindigkeit konnte auch für dieses Verfahren die Reynodls-Zahl bestimmt werden. Bei dieser Methode kam ein Wert von

$$\boxed{Re_{\text{hp}} = (0.1617 \pm 0.0016)} \tag{5.4}$$

heraus. Dass die Reynoldszahelen nicht zusammenzupassen scheinen ist ganz richtig, denn die Reynodlszahl muss für jedes System seperat betrachtet werden.

5.2 Analyse der Messwerte

Nun sollen die Werte noch analysiert werden und überprüft werden, ob die Werte deckungsgleich sind.

Viskosität Zunächst soll geschaut werden, ob die Werte der Viskosität sich statistisch decken können. Dafür wird die Standardabweichung genutzt. Nach der Berechnung kommt man auf eine Abweichung von

$$\sigma_{\text{visk}} = 2.27\sigma \tag{5.5}$$

Damit lässt sich sagen, dass die Ergebnisse zwar noch in einer 3σ-Umgegung liegen und somit im statistisch signifikanten Zusammenhang stehen. Jedoch wären bessere Ergebnisse natürlcih wünschenswert gewesen. Da es leider keinen Literaturwert gibt, können wir hier nicht sagen, welche Methode wirklich die bessere Messmethode darstellt.

Jedoch ist in der ersten Messmethode die menschliche Ungenauigkeit viel mehr von bedeutung, als in der nach Hagen-Poiseuille. Denn in der ersten Methode sind die Zeitintervalle kleiner gewesen, wodurch die Reaktionszeit schwerer wiegt. Zudem war es fast unmöglich, die Kugeln präzise zentrisch ins Sinkrohr einzulassen, dadurch waren die Zeiten noch unpräziser. Zum ausgleich waren fünf Messungen vermutlich nicht genug.

Aber auch die Messmethode nach Hagen-Poiseuille behart auf menschliche Reaktionen. Hier ist die Reaktionszeit jedoch weniger massgeblich. Jedoch ist das Ablesen des Füllstandes eine Herausforderung für das menschliche Auge, da die Oberfläche eine starke Krümmung aufweist und hier das Ablesen vermtulich schwierikger ist, als in der Methode nach Stokes. Auch muss ein konstanter Massefluss sichergestellt sein. Läuft hier etwas schief, so verschieben sich die Werte. So ist es nicht auszuschließen, dass die POM-Kugeln am Boden zu einer leichen Verstopfung füren könnten. Diese Vermutung legt die Abbildung 3.I nahe, da hier unten bereits sehr viele Kugeln liegen und es visuell so scheint, als leigen diese vor der Öffnung.

Dennoch wird die These hier aufgestellt, dass die Methode nach Hagen-Poiseuille genauer ist, wie im Paragraphen „Reynoldszahlen“ erläutert werden soll.

Geschwindigkeiten Doch zunächst sollen noch die Geschwindigkeiten diskutiert werden. Wie hilfreich war die Ladenburg-korrektur und wie weit weichen selbst die korrigierten Sinkgeschwindigkeiten von den theoretischen Werten ab? Die Antworten auf diese Fragen sind tabellarisch bantwortet (Tabelle 5.I).

Tabelle 5.I:
Vergleich und Bestimmugn der Signifikanz der verschiedenen Geschwindigkeiten für die Kugel

$v_{\text{gem}}[\text{m/s}]$	$v_{\text{korr}}[\text{m/s}]$	$v_{\text{theo}}[\text{m/s}]$	$\sigma_{\text{gem, korr}}$	$\sigma_{\text{korr, theo}}$
0.63 ± 0.03	0.641 ± 0.026	0.48 ± 0.05	0.28σ	2.86σ
2.00 ± 0.08	2.06 ± 0.09	1.91 ± 0.18	0.50σ	0.75σ
4.55 ± 0.07	4.74 ± 0.11	4.2 ± 0.4	1.46σ	1.30σ
7.48 ± 0.07	7.90 ± 0.16	7.4 ± 0.7	2.40σ	0.70σ
11.63 ± 0.23	12.4 ± 0.3	11.5 ± 1.0	2.04σ	0.86σ
15.02 ± 0.20	16.3 ± 0.4	16.3 ± 1.5	2.86σ	0.00σ
20.53 ± 0.18	22.5 ± 0.7	21.90 ± 1.10	2.73σ	0.46σ
25.3 ± 0.4	28.15 ± 1.11	28.2 ± 2.6	2.42σ	0.02σ
30.9 ± 0.3	34.8 ± 1.6	35 ± 3	2.40σ	0.06σ

Die Werte zeigen zum einen sehr schön die Beobachtung, die in Abbildung 4.I gemacht wurde, denn die Ladenburg-Korrektur macht sich besonders bei größeren Radien bemerkbar. Zudem wird deutlich, dass selbst die Ladenburg-Korrektur zwar nicht vollständig ausreicht, um die turbulente Strömung wegzukorrigieren, jedoch nähren sich die Werte erstaunlich gut der 0.00σ-Umgebung an. Dies liegt jedoch auch mitunter daran, dass die Fehler immer größer werden und diese immer schwerwiegender werden, da diese in der Qadratsumme stehen und deren Beitrag somit immer signifikanter wird, je größer die Ungenauigkeit wird. Dennoch bleiben die Werte verblüffend nah beieinander.

Reynodlszahl Zuletzt sollen noch die Reynodlszahlen betrachtet werden. Diese untereinander zu vergleichen würde keinen Sinn ergeben, daher wollen wir schauen, wie stark die Werte von den im Skript genannten kritischen Werten abweichen. [Wag26]

Somit kommt man für Stokes auf eine Abweichung von

$$\sigma_{\mathrm{Re,\ s}} = 1.33\sigma\,. \tag{5.6}$$

Wie zu erwarten liegen wir ca. um den kritischen Punkt, denn dieser Punkt sollte gerade abgeschätzt werden. Da aber nun geschätzt wurde, hätte die Signifikanz bei einer guten Schätzung unter 1σ fallen müssen, daher lässt sich sagen, dass die Abschätzung etwas zu liberal war. Ab einem Fehler von 0.4 wäre hier die Signifikanz unter 1.

Für die Werte von Hagen-Poiseuille kommt man auf eine Signifikanz von über $1 \cdot 10^7\sigma$. Dmait ist mehr als eindeutig, dass die Strömung während der Messung als Laminar angenommen werden konnte.

Daraus lässt sich auch deuten, wieso die Methode nach Hagen-Poiseuille vermutlich präziser ist, als die von Stokes; denn die Annährung an die theoretisch laminare Strömung wird hier weitaus besser bewerkstelligt.

5.3 Kritik

Zum Abschluss noch ein paar Kritiken. Als erstes wäre es besser gewesen, wenn der Volumenfluss über die Masse und Dichte des PEG bestimmt werden würde. Dann könnte man anstatt der visuellen Einschätzung eine Waage zur Hilfe nehmen und den Fehler minimieren. Außerdem wäre es vorteilhaft zu wissen, welches PEG es genau ist. Denn die PEG-Kettenlänge in der Industrie geht von gerade mal 100 Kettengliedern hin bis zu tausenden. Daher ist ein Vergleich mit online Werten witzlos. Auch ergibt die Grafik „Dichte von Polyethylenglykol“ erst richtig sinn, wenn das genau PEG bekannt wäre.

6. Phython-Code

Der gesamte Pythoncode ist auf auf meinem GitHub unter https://github.com/FinnZeumer/PAP-2 [Fin26] zu finden. Zudem ist hier auch der Souce-Code für dieses Projekt selbst, falls Interesse besteht diesen zu sehen.

Der Code ist primär selbst geschrieben. Es wird jedoch die in Teilen die eingebaute Funktion des „MS Copilot“ in Visual Studio Code verwendet. Wenn es nicht anders gekennzeichnet wird, wird diese jedoch als Unterstützung genutzt und nicht zum generieren ganzer Methoden. Zudem haben wir einen Ordner mit unseren Code-Projekten in der Freundesgruppe. Hier Teilen wir alle unseren Code. Ich erhebe keinen Ansprch auf alleiniges Schreiben des Codes.

PAP 2 – Python-Auswertung

Wichtige Funktionen für das PAP.

Dieses Dokument ist von mir (FEZ) selbstständig geschreiben und verfasst worden. Viele Funktionen sind von anderen übernommen und abgewandelt worden. Da der Code in VS Code geschrieben wird, wird teilweise auf den MS Copilot (K.I. Modell) zurückgegriffen. Dieser wird unterstützend und nicht generierend benutzt. Ich entschuldige mit hiermit auch formal für das unglaublich schlimme Denglisch in den Kommentaren und der Dokumentation.

Vorlage zum bestimmen aller im Praktikum benötigten Formel und intigriertem Latex Export für eine effizientere Nutzung.

Diese Version ist noch nicht aktiell. Daher führen viele Überschriften in Leere Kapitel. (Stand 07.03.2026)

Inhaltsverzeichnis für den Papulator

- Definition der Versuchsvariablen
- Import aller genutzen Libaries
- Funktionen der Fehlerrechnung
  - Runden signifikanter Stellen
  - Gaussische Fehlerfortpflanzung
  - Berechnung von Messwert und Fehler
  - Signifikante Standartabweichung zweier Messwerte
- Messdatenimporte
  - Patterns
  - Import von Experimentdaten
- Latex Export
  - Export von Gleichungen
  - Export von Einzelergebnissen
  - Export von Messreihen (Tabelle))
- Calc_Everything

- Plotting
- Ergebnisse der Aufgaben

Definition der Versuchsvariablen

Im Folgenden sind Versuchsvariablen definiert, die für einen besseren Workflow sorgen sollen. Diese werden zum exportieren und überschreiben von Dateien wichtig sein und ermöglichen es, diese Datei für jeden Versuch zu benutzen und lediglich die Variablen zu verändern. Es ist jedoch empfohlen eine Kopie der Forlage für jeden Versuch zu machen, damit dieses Dokument strukturiert bleibt.

Zudem sind wichtige Konstanten definiert, die immer wieder auftauchen.

In [3]:
```
versuchsnummer = "212"
versuchsname = "Beispielhafter-Name" # Auf Leerzeichen verzichten
aufgabe = "0z"

# Wichtige Konstanten
# Heidelberger Ortsgravitationsbeschleunigung in m/s^2
g_hd = 9.80984
err_g_hd = 0.00002

# Farben in hex
gruen="#1b722f"
hellgruen = "#1ba97f"
rot = "#b92d3a"
weinrot="#721c24"
blau="#073984"
lila = "#6610f2"
```

Import aller genutzen Libaries

In [4]:
```
# Numpy für bessere Berechnungen
import numpy as np
from numpy import exp, sqrt, log, pi
from uncertainties import unumpy as unp

# Weiteres für bessere Rechnungen
import pylab as py

from decimal import Decimal, ROUND_HALF_UP, getcontext # Besonders für sig. Runden

# Berechnungen und Plotting
from scipy import odr
import scipy.optimize
from scipy.optimize import curve_fit
from scipy.stats import chi2
from scipy.stats import poisson
from scipy.integrate import quad
from scipy.signal import find_peaks
from scipy.signal import argrelextrema, argrelmin, argrelmax
```

```python
from scipy.special import factorial

import matplotlib.pyplot as plt
import matplotlib.mlab as mlab
import matplotlib.transforms as transforms

# Zum Auslesen von Dateien und ähnlichem
import os
import os.path

import pandas as pd # Auch wichtig für den Latex Export
import csv
import re
from typing import List


# Besseres Funktionen handling
import sympy as sp
from sympy import separatevars

# Display und Output
from IPython.display import display, Math, Latex, HTML
```

Funktionen der Fehlerrechnung

Folglich werden eigene Funktionen definiert, die das Auswerten von Versuchen enorm vereinfachen sollen. Diese sind in vielen Punkten an fremden Code orientiert.

Runden signifikanter Stellen

In [5]:
```python
def round_sig_digs(errVal: float, val: float=None):
    """
    Function that rounds error value accoring the the standardised format:
    the function will search for the first digit that is greater than 2. If it found such a dig i
    It will than present the value according to the errVale, thus they have the same amount of di

    Parameter
    ---------
    **errValue** : float to be rounded to significant digit.

    **value** : float that will be rounded accoring to the errValue.

    Returnes
    --------
    **rounded** : Tupel
        Gibt den Fehler und den Messwert gemäß DIN gerundet zurück.
    """

    # ---------------------------------------------------------------------------
    #   Definition einer Hilfsfunktion, damit der Messwert auf die Stellen-
    #   länge des Fehlers angepasst werden kann und 9en keine Rundungs-
    #   probleme darstellen.
    # ---------------------------------------------------------------------------
    def round_with_carry(number_str):
```

```
        number_list = list(number_str)

        i = len(number_list) - 1

        # Solange 9 auf 0 setzen und nach links gehen
        while i >= 0 and int(number_list[i]) > 4:
            number_list[i] = '0'
            i -= 1

        # Wenn wir noch links eine Stelle haben erhöhen
        if i >= 0:
            number_list[i] = str(int(number_list[i]) + 1)
        else:
            # Alle Stellen waren 9 neue 1 vorne
            number_list.insert(0, '1')

        return ''.join(number_list)

    errVal = Decimal(str(errVal))
    val = Decimal(str(val))
    err_str = f"{errVal.normalize():f}"
    val_str = f"{val.normalize():f}"

    if '.' in err_str:
        err_int, err_frac = err_str.split('.')
    else:
        err_int = err_str
        err_frac = ''

    dez_pos = len(err_int) # Position des Kommas
    err_comb = err_int + err_frac + '0' # Alle Ziffern des Fehlers ohne Dezimal-trennung puls ext
    err_diffrent_dig = len(set(list(err_comb))) # Gibt die Anzahl der verschiedenn Ziffern an

    if '.' in val_str:
        val_int, val_frac = val_str.split('.')
    else:
        val_int = val_str
        val_frac = ''

    val_comb = val_int + val_frac


    leadZeros = False
    sig_dig_bool = False
    found_non_sig_dig = False

    fracZeros = 0
    non_sig_dig = ""

    rounded_err_dig = ""
    sig_dig_nine = False

    pos_non_sig_dig = 0

    val_round_up = False


    # Checken, ob Fehler kleiner ist als Messwert
    if errVal > val:
        print("Der Fehler ist größer als der Messwert")
        print("---------------- \n")
        return
```

```python
# ---------------------------------------------------------------------------
#   Untesucht ob und wo die Signifikante ist.
# ---------------------------------------------------------------------------

if err_int == '0':
    leadZeros = True

for numPos, dig in enumerate(err_comb):
    # Schaut ob dig sig ist
    if int(dig) > 2:
        # print(f"{dig} ist Signifikant")
        pos_sig_dig = numPos
        sig_dig_bool = True
        if dig == '9':
            sig_dig_nine = True
        break
    # Schaut ob dig nach dem Komma 0 ist
    elif (int(dig) == 0) and found_non_sig_dig and leadZeros:
        fracZeros = fracZeros + 1
        non_sig_dig = non_sig_dig + dig
    else:
        # print(f"{dig} ist nicht Signifikant")
        pos_non_sig_dig = numPos
        found_non_sig_dig = True
        non_sig_dig = non_sig_dig + dig
        # Schaut ob dig nicht signifikant ist. Dann soll
        if (len(non_sig_dig) - fracZeros) == 2:
            sig_dig_bool = False
            break
        # print(len(non_sig_dig.replace('0','')))
        pass


if leadZeros == True and sig_dig_bool == False:
    if (('1' in err_comb or '2' in err_comb)) and (err_diffrent_dig== 1) or (('1' in err_comb
        # print("Alle Ziffern sind nicht signifikant")
        pass

    elif (('0') in list(err_comb)) and (len(set(list(err_comb)))== 1):
        print("Die Ungenauigkeit ist null.")
        pass

    else:
        # print("Alle stellen sind 0, 1 oder 2 ")
        pass

# ---------------------------------------------------------------------------
#   Untersucht die Folgestelle der Signifikanten Stell: Auf- oder Abrunden?
# ---------------------------------------------------------------------------

# Korrekt runden
if sig_dig_bool == True:
    if (int(err_comb[pos_sig_dig + 1]) < 5):
        rounded_err_dig = int(dig)
        rounded_up = False
    else:
        rounded_err_dig = int(dig)  + 1
        rounded_up = True
```

```python
# 1 oder 2 und nach dem Komma
if sig_dig_bool == False and leadZeros and len(err_frac) > 1:
    if err_comb[pos_non_sig_dig + 1] == '0':
        rounded_err_dig = int(err_comb[pos_non_sig_dig + 1])
        rounded_up = False
    elif (int(err_comb[pos_non_sig_dig + 2]) < 5):
        # print(f"{err_comb[pos_non_sig_dig + 1]} muss abgerundet werden")
        rounded_err_dig = int(err_comb[pos_non_sig_dig + 1])
        rounded_up = False
    else:
        # print(f"{err_comb[pos_non_sig_dig + 1]} muss aufgerundet werden")
        rounded_err_dig = int(err_comb[pos_non_sig_dig + 1])  + 1
        rounded_up = True

    # Wissenschaftliche Schreibweise mit e fixen
    if 'e' in str(errVal):
        res = (str(errVal))
    else:
        rounded_err = non_sig_dig
        a = list(rounded_err)
        a.insert(dez_pos, '.')
        res = (''.join(a))

elif sig_dig_bool == False and leadZeros and len(err_frac) == 1:
   rounded_err_dig = err_frac + '0'
   rounded_up = False


# 1 oder 2 vor dem Komma
if (sig_dig_bool == False) and (leadZeros == False):
    if (int(err_comb[pos_non_sig_dig + 1]) < 5):
        # print(f"{err_comb[pos_non_sig_dig]} muss abgerundet werden")
        rounded_err_dig = int(err_comb[pos_non_sig_dig])
        rounded_up = False
    else:
        # print(f"{err_comb[pos_non_sig_dig]} muss aufgerundet werden")
        rounded_err_dig = int(err_comb[pos_non_sig_dig])  + 1
        rounded_up = True

    # Wissenschaftliche Schreibweise mit e fixen
    if 'e' in str(errVal):
        res = (str(errVal))
    else:
        rounded_err = non_sig_dig
        a = list(rounded_err)
        a.insert(dez_pos, '.')
        res = (''.join(a))

# Neunen richtig runden
elif sig_dig_nine and rounded_up and sig_dig_bool == False:
    rounded_err = round_with_carry(err_int)
    print(rounded_err)
    res = rounded_err

# Nullen ordentlich betrachten
elif leadZeros and err_int == "0" and sig_dig_bool:
    rounded_err = non_sig_dig + str(rounded_err_dig)
    a = list(rounded_err)
    a.insert(dez_pos, '.')
    res = ''.join(a)

# Richtig runden, wenn Stelle vor dem Komma leigt
```

```
    elif dig in err_int:
        rounded_err = non_sig_dig + str(rounded_err_dig) + (len(err_int) - pos_sig_dig - 1) * '0'
        res = rounded_err
    else:
        rounded_err = non_sig_dig + str(rounded_err_dig)
        a = list(rounded_err)
        a.insert(dez_pos, '.')
        res = ''.join(a)


    if err_comb[pos_non_sig_dig + 1] == '9':
        sig_dig_nine = True

    if sig_dig_nine and rounded_up and leadZeros:
        if sig_dig_nine and rounded_up:
            rounded_err = non_sig_dig[:len(non_sig_dig) - 1] + str(int(dig) + 1)
            a = list(rounded_err)
            a.insert(dez_pos, '.')
            res = ''.join(a)
        elif rounded_up:
            rounded_err = non_sig_dig[:len(non_sig_dig)] + str(int(err_comb[len(non_sig_dig)]) +
            a = list(rounded_err)
            a.insert(dez_pos, '.')
            res = ''.join(a)
        else:
            rounded_err = non_sig_dig[:len(non_sig_dig)] + err_comb[len(non_sig_dig)]
            a = list(rounded_err)
            a.insert(dez_pos, '.')
            res = ''.join(a)

    # Runden, wenn die Stelle nicht signifikant ist
    # Nach dem Komma
    elif leadZeros and sig_dig_bool == False:
        if sig_dig_nine and rounded_up:
            rounded_err = non_sig_dig[:len(non_sig_dig) - 1] + str(int(dig) + 1) + '0'
            a = list(rounded_err)
            a.insert(dez_pos, '.')
            res = ''.join(a)
        elif rounded_up:
            rounded_err = non_sig_dig[:len(non_sig_dig)] + str(int(err_comb[len(non_sig_dig)]) +
            a = list(rounded_err)
            a.insert(dez_pos, '.')
            res = ''.join(a)
        else:
            rounded_err = non_sig_dig[:len(non_sig_dig)] + err_comb[len(non_sig_dig)]
            a = list(rounded_err)
            a.insert(dez_pos, '.')
            res = ''.join(a)

    # Vor dem Komma
    if (leadZeros == False) and (sig_dig_bool == False):
        if rounded_up:
            rounded_err = err_int[0] + str(rounded_err_dig) + '0' * (len(err_int) - 2)
            a = list(rounded_err)
            a.insert(dez_pos, '.')
            res = ''.join(a)
        # elif (rounded_up == False) and (len(err_int) == 2):
        #     rounded_err = non_sig_dig[:len(non_sig_dig)]
        #     a = list(rounded_err)
        #     a.insert(dez_pos, '.')
        #     res = ''.join(a)
        else:
```

```
            rounded_err = non_sig_dig[:len(non_sig_dig)] + '0' * (len(err_int) - 2)
            a = list(rounded_err)
            a.insert(dez_pos, '.')
            res = ''.join(a)

        if int(val_int[abs(len(err_int) - len(val_int)) + 1]) < 5:
            val_round_up = False
        else:
            val_round_up = True


    # ---------------------------------------------------------------------------
    #   Anpassen des Messwertes an den gerundeten Fehler (Gleich gerundet)
    # ---------------------------------------------------------------------------

    last_zero = False
    if res[len(str(res)) - 1] == "0":
        last_zero = True

    val = Decimal(str(val))
    res = Decimal(str(res))

    # Exponent (z.B. -2 bei 0.12, +2 bei 200)
    num_decimals = -res.normalize().as_tuple().exponent

    # Rundungsstelle bestimmen
    if (leadZeros == False):
        if val_round_up:
            rounded_val = val_int[:abs(len(err_int) -  len(val_int))] + str(int(val_int[abs(len(e
        else:
            rounded_val = val_int[:abs(len(err_int) -  len(val_int))] + str(int(val_int[abs(len(e
    elif sig_dig_bool:
        rounded_val = val.quantize(Decimal('1.' + '0' * (num_decimals)), rounding=ROUND_HALF_UP)
    elif last_zero:
        rounded_val = Decimal(str(val.quantize(Decimal('1.' + '0' * (num_decimals)), rounding=ROU
    else:
        rounded_val = Decimal(str(val.quantize(Decimal('1.' + '0' * (num_decimals)), rounding=ROU

    # Debugging Print
    print(f"Aus den Messwert {val} mit der Ungenauigkeit {errVal} berechnet sich das Ergebnis zu:
    print("---------------- \n")

    return rounded_val, res
```

Gaussische Fehlerfortpflanzung

In [6]:
```
def gff(func, errPronePar):
    """
    Kann die Fehlerformel einer gegebenen Gleichung bestimmen.

    Parameters
    ----------
    **func** : sympy function
        Funktion dessen Fehler bestimmt werden soll.

    **errPronePar** : Array
        Liste (Array) aller fehlerbehafteten Größen der Gleichung con sp.Symbols
        Diese Werte werden als x_sym, y_sym, z_sym etc. bezeichnet und sind ungleich den Werten f
        Für die Werte wird daher die Bezeichnung x_val, y_val, z_val etc. genutzt und für deren F
```

```
    Return
    ----------
    **absolut_err** : sympy function
        Gibt die Fehlergleichung des absoluten Fehlers wieder.

    **relativ_err** : sympy function
        Gibt die Fehlergleichung des relativen Fehlers wieder.

    **errProneParamters** : array
        Liste aller Fehlerbehafteten Größen
    """

    error = 0
    errProneParamters = []
    for errPar in errPronePar:
        d = sp.symbols('thisWillTurnDelta' + errPar.name)
        partial = sp.diff(func, errPar) * d  # Die Funktion wird nach der fehlerbehafteten Variab
        error = error + partial**2 # Fehler werden quadratisch aufsummiert
        errProneParamters.append((errPar,d))
    absolut_err=sp.simplify(sp.sqrt(error),rational = True)
    relativ_err=sp.simplify(sp.sqrt(error/func**2),rational = True)

    # Debugging Print
    # print(f"Die Fehlerfunktion zur Ausgangsfunktion {func} wurde erfolgreich nach den Fehlergrö

    return absolut_err, relativ_err, errProneParamters
```

Berechnung von Messwert und Fehler

In [7]:
```
def calc_with_err(func, errFunc, values):
    """
    Methode zum berechnen von Werten und deren Fehler.

    Parameter
    ----------
    **func** : sympy function
         Funktionen mit Parametern.

    **errFunc** : sympy function
        Die zu func gehörende Fehlerformel. Wird über gff bestimmt.

    **values** :
        Werte, die in die Funktionen eingesetzt werden.
        Als array von Tupeln der Form [(a,da),(b,db),...] oder als array/liste [a,da,b,db,...]
        (Reihenfolge muss die sein, in der die Argumente in der Funktion genommen werden)

    Return
    ----------

    """

    #Falls der Input in mehrere Tupel aufgeteilt ist, werden diese zu einem Array zusammengefügt
    if (np.ndim(values) != 1):
        values = np.concatenate(values)
    result = func(*values[::2])
    uncertainty = errFunc(*values)

    # Debigging Print
    # print(f"Die Ergebnisse wurden erfolgreich durch Funktion und Fehlerfunktion bestimmt!")
```

```
    print(f"{result} \\pm {uncertainty}")

    return result, uncertainty
```

Fehlerfunktion und Messwerte

Signifikante Standartabweichung zweier Messwerte

In [8]:
```
def sigma_abweichung(p1, p2, err_p1 = 0, err_p2 = 0):
    """
    Funktion zum berechnen der Sigma-Abweichugn von zwei Messwerten, oder einem Messwert und eine
    """
    if err_p1 == err_p2 == 0:
        print("Für die Sigma-Abweichung muss mindestens ein Wert fehlerbehaftet sein!")
    else:
        abweichung = abs(p1 - p2)/(np.sqrt(err_p1 ** 2, err_p2 ** 2))
    return abweichung
```

Messdatenimporte

Für die Auswertung von Versuchen ist es nicht selten, dass Messwerte in *CSV* oder in *TXT* Dateien gespeichert sind. Daher sollen diese Vernünftig eingelesen und verarbeitet werden, um das Benutzen der Daten möglichst einfach und instuinktiv zu gestalten.

Um das Handeln von CSV Tabellen möglichst einfach zu gestalten, werden RIGEX-Pattern genutzt. Das heißt, wenn die Tabelle vor einem Versuch ordentlich strukturiert ist, können die Werte sofort genutzt werden, so wie es gewünscht ist. Dabei sollen folgende Eigenschaften erfüllt sein:

- Das Nutzen einer Kopfzeile ist nicht nur möglich, sondern explizit empfohlen.
- Erkennung eines Messwertes, seiner Einheit und einer Potenz. Das hat dan bspw. die Form: *(name [unit] 10^{power})*
- Der zum Messwert gehörende Fehler. Dies ist meist ein Ablese Fehler. Die Bennenung sollte dabei: *err name* sein.
- Direkte Auswerten von Messreihen. Soll bspw. über mehrere Werte gemittelt werden, so wird automatisiert Mittelwert, Standartabweichung und gesamt Fehler der Messreihe bestimmt.
- Dimensionslose Größen sollen erkannt werden.
- Einstellung eines Indexes. Dies kann entweder die nummerische Angabe des Messdurchganges sein, oder der Parameter-Name. Wird bspw. dieselbe Messung in Wasser und in Luft durchgeführt, so sollen dies die Parameter-Namen sein. Diese sollen in der ersten Spalte der Tabelle stehen.

Patterns

In [9]:
```
# RegEx Patterns, um autoamtisiert Schemata anzuwenden und Daten anhand dieser auszuwerten.

# Finden eines Messwertes, seiner Einheit und seiner Potenz
pattern = re.compile(
        r'^\s*'
```

```
        r'(?P<name>[^\[]+?)'    # alles bis zur ersten '['      # Name
        r'\s*\[\s*'             # echtes '['
        r'(?P<unit>[^\]]+?)'    # alles bis zur ']'             # Einheit
        r'\s*\]'                # echtes ']'
        r'(?:\s*10\^(?P<power>-?\d+))?'                         # optionale 10^n
        r'\s*$'
    )

series_pattern = series_pattern = re.compile(
        r'^(?P<base>\w+)_\d+'
        r'(?:\s*\[\s*(?P<unit>[^\]]+)\s*\])?'
    )

# Index Spalte idx
index_pattern = re.compile(
    r'idx\s*'
    )

# Spalte die Ignoriert werden sill ign
ignore_pattern = re.compile(
    r'(?:ign[^,]*\s*,\s*)?'
)

# Messwert mit Einheit
messurment_pattern = re.compile(
    r'(?P<value>Messwert)\s*$$(?P<unit>[^$$]+)\]\s*,\s*'
)

# Fehler des Messwertes
error_pattern = re.compile(
    r'(?:err\s+Messwert\s*,\s*)?'
)
```

Import von Experimentdaten

Das einlesen von Messweerten und das speichern dieser als Arrays für einen besseren zugriff. Automatisches herausfinden von Messwert und dazugehörigem Fehler.

In [10]:
```
def import_experimental_data(path:str = versuchsnummer + ".csv", setDelimiter:str = ",", setHeade
    """

    Parameter
    ---------
    **path**: str
        Relatives verzeichnes der einzulesenden Datei. Am besten CSV-Datei einfach unter *versuch

    **setDelimiter**: str
        Default ist hier das Komma:  > , <

    **setHeader**: int
        Setzt fest, was die Header-Row ist. Default ist 0 (die oberste Row)

    Return
    ---------
    **cleanded** : Pandas DataFrame
        DataFrame, welches sicher zum weiterverwenden ist. NaN und ähnliches werden herausgefilte
    """

    # Liest die Daten unmaipuliert
```

```
    try:
        df = pd.read_csv(path, delimiter=setDelimiter, header=0, index_col=None)
    except FileNotFoundError:
        print(f"The file {path} was not found.")

    # Wir erwarten nicht, dass zwei perfekt identische Zeilen existieren koennten, daher werden d
    # Langfristig soll das noch verschönert werden und wirklich nur NaN Zeilen gelöscht werden
    no_NaN_data = df.drop_duplicates(keep= False)
    df.reset_index(drop=True, inplace=True)

    return no_NaN_data

# Speichert direkt das DataFrame als Variable, so kann dieses direkt in anderen Cells benutzt wer
experimental_data = import_experimental_data()
```

Latex Export

Erstellen der dedizierten Dartei zum Speichern der Werte für **LATEXT**

Folgender Code sollte einmal ausgeführt werden, damit direkt die Datei zum Speichern erstellt werden kann. Unbedingt daran denken, die Versuchsnummer anzupassen, die Werte anderer Versuche könnten ansonsten verloren gehen. Es gibt jedoch eine sicherheits Kopie.

Die File wird Folgendes beinhalten:

- Formel, sowie Formel nach gff
- Berechnete Werte und deren Fehler
  - Wert + Fehler
  - Tabellen Export

In [11]:
```
def create_tex_result_file(fileName:str = f"python-results.sty", relativePath:str="../Auswertung/
    """
    Diese Method erstellt automatisch eine tex-Datei, in dem Messwerte bzw. deren Ergebnisse, Tab
    Die File wird unter *Versuche/${versuchsnummer}$/Auswertung/python-results.tex* zufinden seir
    angepasst werden muss.

    Die Python-File muss im Python-Ordner des jeweiligen Versuches liegen!

    Parameter
    ----------
    **fileName** :
        neuer Name, falls die Datei besonders heißen soll.

    Return
    --------
    **path** : str
        Gibt den Path zur file wieder, welche autoamtisch generiert. In diese sollen später die F
    """

    path = relativePath + fileName

    if os.path.isfile(path):
        print(f"{fileName} existiert bereits.")
```

```
        pass
    else:
        print(f"{fileName} wird in {relativePath} erzeugt.")
        with open(path, 'w') as file:
            file.write("% Dies ist eine automatisch generierte Datei. Hier werden automatisiert \

        with open(path + "-BackUp.sty", 'w') as bFile:
            bFile.write("% Dies ist eine automatisch generierte Datei. Hier wird dediziert ein Ba

    return path

# pyPath ist die File, in die der Latex Export geschehen soll.
pyPath = create_tex_result_file()
```

```
python-results.sty existiert bereits.
```

Export von Gleichungen

In [12]:

```
def function_to_latex(func, texVarName:str=None, texCom:str=None):
    """
    Zeigt die Formel als gerenderte Math-Darstellung und darunter
    den Latex-Quelltext, der per Button kopiert werden kann. (Für leichtere Benutzung als HTML).

    Zudem wird die Latexformel als Variable in Latex gespeichert.

    Parameters
    ----------
    **f** : sympy function

    **texVarName** : str
        Einzigartiger Name der Variablen. Dieser wird zum überschreiben alter Formel gebraucht.

    **texCom** : str
        setzt den newcommand-Kürzel für latex fest. Setzte kein Backslash! Auch texCom muss einzi
    """
    print("Die gegebene Funktion lautet: \n")
    display(Math(sp.latex(func, long_frac_ratio=2).replace('thisWillTurnDelta', r'\Delta '))

    print("Hier ist der dazugehörige Latex code: \n")
    latex_str = sp.latex(func, long_frac_ratio=2).replace('thisWillTurnDelta', r'\Delta ')
    html = f"""
    <div style="margin-top:0.5em;">
        <code id="latex-code-{id(func)}">
            {latex_str}
        </code>
        <button onclick="
            const tex_as_txt = document.getElementById('latex-code-{id(func)}').innerText;
            navigator.clipboard.writeText(tex_as_txt)
        " style="
            margin-left:8px;
            padding:2px 6px;
            cursor:pointer;
        ">
            Kopieren
        </button>
    </div>
    """
    display(HTML(html))


# ---------------------------------------------
```

```
#   Diese Funktion hat leider einen Bug.
# ------------------------------------------

    # # Hinzufuegen bzw. Ueberschreieben der Formel in die Sammlung
    # with open(pyPath, 'r') as file:
    #     lines = file.readlines()
    # found = False

    # with open(pyPath, 'r') as file:
    #     for lineNum,  line in enumerate(lines, 1):
    #         if texVarName in line: # Checkt, ob der Variablen Name bereits vergeben ist.
    #             lines[lineNum] = "\\newcommand{\\" + texCom + "}{" + latex_str + "} \n\n"
    #             found = True
    #             # Deugging print
    #             # print(f'{texVarName} is at line {lineNum}')

    # if not found:
    #     print("Die neue Funktion wurde hinzugefügt")
    #     with open(pyPath, 'w') as file:
    #         file.writelines(lines) # Schreibt den alten Stand
    #         file.write("\n\n% " + texVarName + "\n") # Fügt die Variable als Kommentar hinzu
    #         file.write("\\newcommand{\\" + texCom + "}{" + latex_str + "} \n\n")
    # else:
    #     print("Die alte Funktion wurde erfolgreich überschrieben.")
    #     with open(pyPath, 'w') as file:
    #         file.writelines(lines)
```

Export von Einzelergebnissen

Export von Messreihen (Tabelle)

In [13]:
```
def table_to_latex(df_tex: str, texVarName: str, texCom: str):
    """
    Fügt aus einem DataFrame entstandene Latex Tabelle der special Python File hinzu.

    Parameters
    ----------
    **df_tex** : pd.DataFrame.to_latex
        Uebersetztes DataFrame. Kann Label und Caption handlen. Dezimaltrennung steht auf ','.

    **texVarName** : str
        Einzigartiger Name der Variablen. Dieser wird zum überschreiben alter Formel gebraucht.

    **texCom** : str
        setzt den newcommand-Kürzel für latex fest. Setzte kein Backslash! Auch texCom muss einzi

    Returns
    -------
    Schreibt automatisch in die sty file
    """

    # Hinzufuegen bzw. Ueberschreieben der Tabelle in die Sammlung
    with open(pyPath, 'r') as file:
        lines: List[str] = file.readlines()
    found = False

    with open(pyPath, 'r') as file:
        for lineNum,  line in enumerate(lines):
```

```python
        if texVarName in line: # Checkt, ob der Variablen Name bereits vergeben ist.
            print(f'{texVarName} starts at line {lineNum + 1}')
            found = True
            table_start_line = lineNum
            break

    if not found:
        print(f"Die neue Tabelle ({texVarName}) wurde hinzugefügt")
        with open(pyPath, 'w') as file:
            file.writelines(lines) # Schreibt den alten Stand
            file.write("\n% " + texVarName + "\n")
            file.write("\\newcommand{\\" + texCom + "}{" + df_tex + "}\n\n")
    else:
        print(f"Die alte Tabelle ({texVarName}) wird überschrieben.")
        start_table = table_start_line + 1
        while start_table < len(lines) and lines[start_table].strip() == "":
            # Leere Zeile überspringen - das ist unser Marker
            start_table += 1
            break   # Wir wollen nur die erste leere Zeile

        # Falls kein leerer Marker gefunden wurde, gehen wir davon aus,
        # dass die Tabelle direkt in der nächsten Zeile startet.
        if start_table >= len(lines):
            start_table = table_start_line + 1

        end_table = start_table

        while end_table < len(lines):
            cur = lines[end_table].strip()

            # Abbruchbedingungen:
            if (
                cur.startswith(r"\\newcommand")
                or cur.startswith(r"\\section")
                or cur.startswith(r"\\subsection")
                or cur.startswith(r"\\begin")
            ):
                break

            end_table += 1

        if end_table > start_table:
            del lines[start_table - 1:end_table]

        insertion_point = table_start_line + 1  # Direkt nach der Zeile mit texVarName
        new_block = [
            # "\n\n",
            f"% {texVarName}\n",
            f"\\newcommand{{\\{texCom}}}{{{df_tex}}}\n",
        ]
        lines[insertion_point:insertion_point] = new_block

        with open(pyPath, 'w') as file:
            file.writelines(lines)
```

Finaler Export

Für die Auswertung sollen Messwerte und Fehler in einer Zelle stehen. Dies tut *tab_merge_val_err* automatisiert.

In [14]:
```python
def tab_merge_val_err(df: pd.DataFrame, show_series_values:bool=False, setTexTabCap: str="", setT
    """Sucht im DataFrame nach einem Asdruck:
        "<name> [<unit>] <power>"
    und dem dazugehörigem Fehlerausdruck:
        "err <name>".
    Erstellt ein neues DataFrame für den Latex-Export

    Parameters
    ----------
    **df* : pd.DataFrame
        Original data frame.

    **show_series_values** : bool
        Rather the table should show the experimental data or just its mean


    Returns
    -------
    pd.DataFrame
        Eine Kopie von ``df``.
    """

    # Kopiert das alte DataFrame
    df_tex = pd.DataFrame(index=df.index)


    def format_tex(row):
        val = row[col]
        err = row[err_col]
        # return f'({val} \\pm {err}) \\mathrm{{{header_unit}}}'
        val_r, err_r = round_sig_digs(err, val)
        return f'${val_r} \\pm {err_r}$'

    # ---------------------------
    # 1) Einzelmessungen
    # ---------------------------

    for col in df.columns:
        col_match = pattern.match(col)

        if not col_match:
            # No header that follows the expected schema – skip it
            # print(f'Skipping column {col!r}: not "<value> [<unit>] <power>"')
            continue

        # Extract the captured parts
        name = col_match.group('name').strip()
        unit = col_match.group('unit').strip()
        power = col_match.group("power")
        # print(f'Found measurement column: name={name!r}, unit={unit!r}, power={power!r}')

        # Build the name of the error column that must exist
        err_col = f'err {name}'
        if err_col not in df.columns:
            # print(f'No error column "{err_col}" for measurement "{name}" - skipping')
            continue

        if power:
            header_unit = f"$\\mathrm{{{unit}}} \\cdot 10^{{{power}}}$"
        else:
            header_unit = f"$\\mathrm{{{unit}}}$"
```

```python
    df_tex[f'{name} [{header_unit}]'] = df.apply(format_tex, axis=1)


# ----------------------------
# 2) Messreihen
# ----------------------------
grouped = {}

for col in df.columns:
    series_match = series_pattern.match(col)
    if series_match:
        base = series_match.group("base")
        unit = series_match.group("unit")
        # power = col_match.group("power")
        grouped.setdefault(base, {"cols": [], "unit": unit})
        grouped[base]["cols"].append(col)


# --- Berechnungen ---
for base, info in grouped.items():
    cols = info["cols"]
    series_unit = info["unit"] or ""


    err_col = f"err {base}"
    if err_col not in df.columns:
        continue

    values = df[cols]

    if show_series_values == True:
        # Einzelwerte in DataFrame hinzufuegen
        for ind, col in enumerate(cols, start=1):
            df_tex[f'{base}_{ind} [{unit}]'] = df.apply(format_tex, axis=1)

    # Mittelwert
    mean = values.mean(axis=1)

    # Standardabweichung
    std = values.std(axis=1, ddof=1)

    # Fehler des Mittelwerts
    stat_err = std / np.sqrt(len(cols))

    # systematischer Fehler
    sys_err = df[err_col]

    # Gesamtfehler
    total_err = np.sqrt(stat_err**2 + sys_err**2)

    # if power:
    #     header_unit = f"\\mathrm{{{unit}}} \\cdot 10^{{{power}}}"
    # else:
    #     header_unit = f"\\mathrm{{{unit}}}"

    formatted = []

    # for m, e in zip(mean, total_err):
    #     m_r, e_r = round_sig_digs(e, m)
    #     formatted.append(f"${m_r} \\pm {e_r}$")
```

```
        df_tex[f"$\\overline{{{base}}} [\\mathrm{{{series_unit}}}]$"] = formatted

    setTexTabCap="Dies ist die Caption der Tabelle"
    setTexTabLab="bspTab"

    table_to_latex(df_tex.to_latex(decimal=',', caption=setTexTabCap, label=setTexTabLab, positic

    print(df_tex.to_latex(decimal=',', caption=setTexTabCap, label=setTexTabLab, position="h"))
    return df_tex.to_latex(decimal=',', caption=setTexTabCap, label=setTexTabLab, position="h")


# combine_value_error(clean_data)
```

Berechnen neuer Spalten mit Fehler

Die folgende Funktion berechnet anhand einer gegebenen Formel und den gegebenen Parametern die Ergebnisse einer neuen Spalte mit Ihren Fehlern.

Calc_everything

In [15]:
```
def calc_everything(function, params, data, params_without_error=[], print_formula:bool=True):
    """
    Returned Berechneten Wert plus Fehler. Zudem werden die Formeln als Latex-Code ausgeworfen.

    Parameter
    ----------
    **function**: sympy Funktion mit Parametern. In diese werden die Messwerte eingesetzt.

    **params**: Parameter der Funktion. Diese werden als Array von Sympy-Symbolen gebraucht. Bspw

    **data**: 2D-Array mit den Messdaten, sodass die Zeilen die Form haben: [Parameter 1, Fehler
        Die Funktion wird zeilenweise angewandt. Wird kein Fehler für einen Parameter angenommen,
        an die Funktion gegeben werden oder ganz weggelassen werden. Dann muss allerdings der bet

    **params_without_error**: Alle Parameter zu denen kein Fehler explizit in den Daten angegeber
        auch nicht in der Latex Form der Fehlerformel vor

    """

    exp_data = np.zeros((data.shape[0],data.shape[1]+len(params_without_error)))
    i = 0      # läuft durch die Parameter
    j = 0      # läuft durch die expanded data
    z = 0      # läuft durch die eingegebene data, also die Messwerte und deren Fehler
    # Läuft durch jeden Parameter und seinen Fehler
    while (i < len(params)):
        # Checkt, ob der Parameter Fehlerbehaftet ist. Wenn, dann wird an j-ter Stelle des exp_da
        if (params[i] in params_without_error):
            exp_data[:,j] = data[:,z]
            i = i + 1
            j = j + 2
            z = z + 1
        else:
            exp_data[:,j] = data[:,z]
            exp_data[:,j+1] = data[:,z+1]
            i = i + 1
```

```
            j = j + 2
            z = z + 2
    # print(exp_data)

    # Create variable that stores parameters that have no assigned uncertainty
    params_with_error = []
    j = 0
    for n in np.arange(0,len(params)):
        if not (params[n] in params_without_error):
            params_with_error.append(params[n])
            j = j + 1

    # Get the given function and error function as numpy functions
    f = sp.lambdify(params, function, "numpy")
    absolut_err, relativ_err, parameters = gff(function,params) # Gauss Fehlerfortpflanzung
    err_abs = sp.lambdify(np.concatenate(parameters), absolut_err, "numpy")

    # f = sp.lambdify(params,function)
    # err_abs = sp.lambdify(np.concatenate(parameters),absolut_err)

    # Calculate the results for each row of data
    results = np.zeros((data.shape[0],2))
    for n in np.arange(0,data.shape[0]):
        results[n,:] = calc_with_err(f, err_abs, exp_data[n,:])
        # print(calc_with_err(f, err_abs, exp_data[n]))

    # if (len(results) < 10):
    #     print("Results:")
    #     print(results)

    # Substitutes 0 for the uncertainty of the parameters without error, so it doesnt show up in
    for p in params_without_error:
        absolut_err = absolut_err.subs('thisWillTurnDelta'+p.name,0)
    for p in params_without_error:
        relativ_err = relativ_err.subs('thisWillTurnDelta'+p.name,0)

    # Wiedergabe des Latex-Codes
    function = sp.simplify(function,symbols = params, rational= True)
    function = sp.separatevars(function)

    if print_formula:
        print("gegebene Funktion:")
        function_to_latex(function)

        print("Formel des absoluten Fehlers der gegebenen Funktion:")
        function_to_latex(absolut_err)

        print("Formel des relativen Fehlers der gegebenen Funktion:")
        function_to_latex(relativ_err)

    return(results)
```

Plotting

Ergebnisse der Aufgaben

Aufgabe 1

Bestimmung der Viskosität nach Stokes mit einem Kugelfallviskosimeter

In [16]:
```
aufgabe = "1"
```

Definieren der Werte aus der eingelesenen CSV

In [80]:
```
df = experimental_data

# Radius Quadrat
rSqr_val = (df["r [mm]"])**2
rSqr_err = df["d [mm]"] * df["err d"]/2

# Durchschnittliche Zeit pro Messung
t_avg = []
t_err = df["err t"]
for idx, row in df.iterrows():
    # Beispiel: Ausgabe aller t-Werte dieser Zeile
    t_vals = row[['t_1 [s]', 't_2 [s]', 't_3 [s]', 't_4 [s]', 't_5 [s]']]
    # print(f"Messung {idx+1}: t-Werte = {t_vals.values}")
    avg = np.average(t_vals.values)
    t_avg.append(avg)
    # print(f"Durchscnittliche Zeit der Messung{idx+1}: {avg} \n -------")

s_val = df["s [mm]"]
s_err = df["err s"]

# Durchschnittliche Geschwindigkeit pro Messung
v_val = []
v_err = []
for idx, row in df.iterrows():
    val = "s [mm]"
    err = "err s"
    ges_val = df.iloc[idx:idx+1][val]/t_avg[idx]
    ges_err = np.sqrt((s_err[idx] / t_avg[idx])**2 + (s_val[idx] * t_err[idx] / (t_avg[idx])**2)*
    v_val.append(ges_val)
    v_err.append(ges_err)

# Lambda Korrektur
lam_val = []
lam_err = []
for idx, row in df.iterrows():
    val = "r [mm]"
    err = "err d"
    l_val = 1 + 2.1 * df.iloc[idx:idx+1][val]/(75)
    l_err = 2.1 * df.iloc[idx:idx+1][err]/(75)
    lam_val.append((l_val))
    lam_err.append((l_err))

# Korrigierte Geschwindigkeit v_s
vs_val = []
```

```python
vs_err = []
for idx, row in df.iterrows():
    s_val = (v_val[idx] * lam_val[idx])
    s_err = np.sqrt((v_val[idx] * lam_err[idx])**2 + (v_err[idx] * lam_val[idx])**2)
    vs_val.append((s_val))
    vs_err.append(s_err)

print("Wert:")
print(np.array(vs_val))
print("Fehler:")
print(np.array(vs_err))
```

```
Wert:
[[ 0.64144737]
 [ 2.05764612]
 [ 4.73636364]
 [ 7.89631107]
 [12.44186047]
 [16.27627628]
 [22.54620123]
 [28.15189873]
 [34.75308642]]
Fehler:
[[0.02577695]
 [0.08594693]
 [0.11469452]
 [0.15829558]
 [0.33362719]
 [0.40122953]
 [0.73026621]
 [1.10564691]
 [1.64618644]]
```

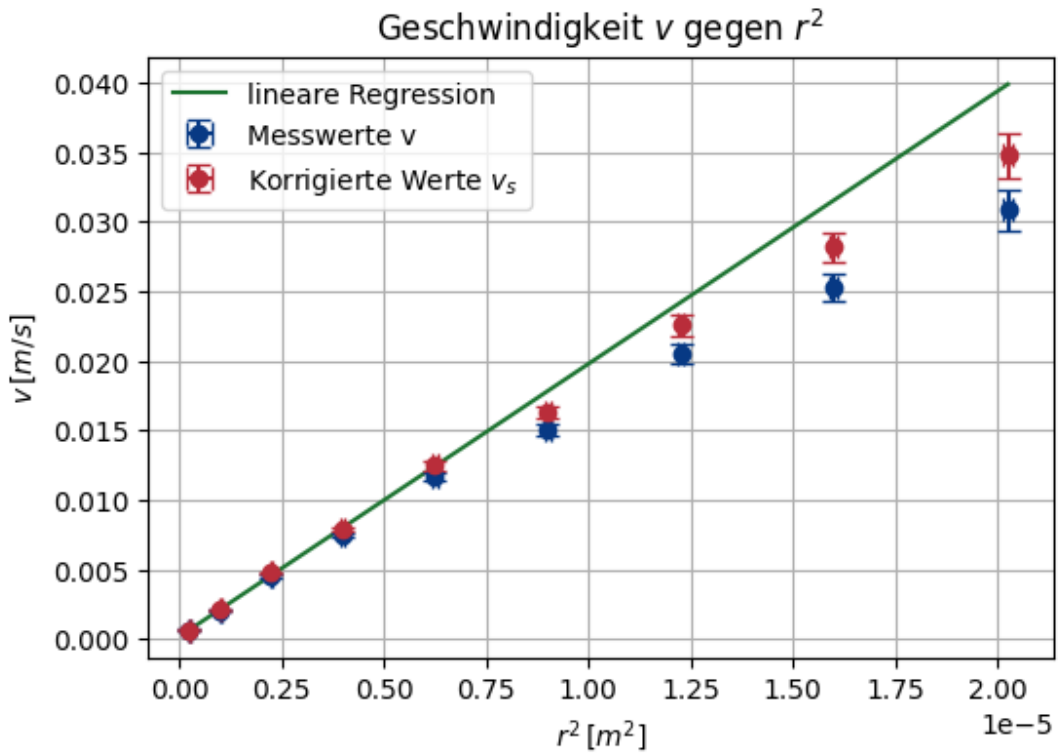
In [18]:

```python
# Einheiten umrechnen

# r^2
x = rSqr_val.values * 1e-6
x_err = rSqr_err.values * 1e-6

# v
y = np.array([float(v) for v in v_val]) * 1e-3
y_err = np.array([float(v) for v in v_err]) * 1e-3

# vs
ys = np.array([float(vs) for vs in vs_val]) * 1e-3
ys_err = np.array([float(vs) for vs in vs_err]) * 1e-3

# Fehlergerade
x_fit = x[:5]
y_fit = ys[:5]

p, cov = np.polyfit(x_fit, y_fit, 1, cov=True)

k, b = p

k_err = np.sqrt(cov[0,0])
b_err = np.sqrt(cov[1,1])

print(f"Steigung k = ({k:.3e} \\pm {k_err:.1e})")
print(f"Achsenabschnitt b = ({b:.3e} \\pm {b_err:.1e})")

x_line = np.linspace(min(x), max(x), 100)
```

```python
y_line = k * x_line + b

# plot
plt.figure(figsize=(6,4))
plt.plot(x_line, y_line, color=gruen, label="lineare Regression")

plt.errorbar(
    x, y,
    xerr=x_err,
    yerr=y_err,
    fmt='o',
    capsize=4,
    color=blau,
    label=r"Messwerte v"
)

plt.errorbar(
    x, ys,
    xerr=x_err,
    yerr=ys_err,
    fmt='o',
    capsize=4,
    color=rot,
    label=r"Korrigierte Werte $v_s$"
)

plt.xlabel(r"$r^2\,[m^2]$")
plt.ylabel(r"$v\,[m/s]$")
plt.title("Geschwindigkeit $v$ gegen $r^2$")
plt.grid(True)
plt.legend()

plt.savefig("../img/plot/ges_gegen_r2.pdf", dpi=300)
plt.show()
```

```
Steigung k = (1.961e+03 \pm 2.5e+01)
Achsenabschnitt b = (1.613e-04 \pm 8.7e-05)
```

```
C:\Users\finnz\AppData\Local\Temp\ipykernel_5428\152146680.py:8: FutureWarning: Calling float on
a single element Series is deprecated and will raise a TypeError in the future. Use float(ser.ilo
c[0]) instead
  y = np.array([float(v) for v in v_val]) * 1e-3
C:\Users\finnz\AppData\Local\Temp\ipykernel_5428\152146680.py:12: FutureWarning: Calling float on
a single element Series is deprecated and will raise a TypeError in the future. Use float(ser.ilo
c[0]) instead
  ys = np.array([float(vs) for vs in vs_val]) * 1e-3
C:\Users\finnz\AppData\Local\Temp\ipykernel_5428\152146680.py:13: FutureWarning: Calling float on
a single element Series is deprecated and will raise a TypeError in the future. Use float(ser.ilo
c[0]) instead
  ys_err = np.array([float(vs) for vs in vs_err]) * 1e-3
```



Steigung bestimmen:

In [19]:
```
vs_sym, r_sym = sp.symbols('v, r')
params = [vs_sym, r_sym]

k = vs_sym / r_sym**2

abs_err, rel_err, pa = gff(k, params)
function_to_latex(abs_err, texVarName="Steigungs Fehler", texCom="k_err")

# values = []
# calc_with_err(k, abs_err, values)
```

Die gegebene Funktion lautet:

$$\sqrt{\frac{1}{r^6}\left(r^2\Delta v^2 + 4\Delta r^2 v^2\right)}$$

Hier ist der dazugehörige Latex code:

```
\sqrt{\frac{1}{r^{6}} \left(r^{2} \Delta v^{2} + 4 \Delta r^{2} v^{2}\right)}
```

Kopieren

Bestimmung der Viskosität

In [20]:
```
g_sym, pk_sym, pf_sym, k_sym = sp.symbols('g, pk, pf, k')
params=[g_sym, pk_sym, pf_sym, k_sym]

eta = 2/9 * g_sym * (pk_sym - pf_sym) / k_sym

values = np.array([[g_hd, err_g_hd, 1385, 15,1148.2, 1.1, 1961, 25]])

calc_everything(eta, params, values, [])
```

```
round_sig_digs(0.017053146577525387, 0.2632409895178194)
```

```
0.2632409895178194 \pm 0.017053146577525387
gegebene Funktion:
Die gegebene Funktion lautet:
```

$$\frac{2g}{9k}(-pf + pk)$$

```
Hier ist der dazugehörige Latex code:
```

```
\frac{2 g}{9 k} \left(- pf + pk\right)
```

Kopieren

```
Formel des absoluten Fehlers der gegebenen Funktion:
Die gegebene Funktion lautet:
```

$$\frac{2}{9}\sqrt{\frac{1}{k^4}\left(g^2\Delta k^2(pf - pk)^2 + k^2\left(g^2\Delta pf^2 + g^2\Delta pk^2 + \Delta g^2(pf - pk)^2\right)\right)}$$

```
Hier ist der dazugehörige Latex code:
```

```
\frac{2}{9} \sqrt{\frac{1}{k^{4}} \left(g^{2} \Delta k^{2} \left(pf - pk\right)^{2} + k^{2} \left(g^{2} \Delta pf^{2} + g^{2} \Delta pk^{2} + \Delta g^{2} \left(pf - pk\right)^{2}\right)\right)}
```

Kopieren

```
Formel des relativen Fehlers der gegebenen Funktion:
Die gegebene Funktion lautet:
```

$$\sqrt{\frac{\Delta pf^2}{(pf - pk)^2} + \frac{\Delta pk^2}{(pf - pk)^2} + \frac{\Delta k^2}{k^2} + \frac{\Delta g^2}{g^2}}$$

```
Hier ist der dazugehörige Latex code:
```

```
\sqrt{\frac{\Delta pf^{2}}{\left(pf - pk\right)^{2}} + \frac{\Delta pk^{2}}{\left(pf - pk\right)^{2}} + \frac{\Delta k^{2}}{k^{2}} + \frac{\Delta g^{2}}{g^{2}}}
```

Kopieren

```
Aus den Messwert 0.2632409895178194 mit der Ungenauigkeit 0.017053146577525387 berechnet sich das
Ergebnis zu:
 0.263 \pm 0.017
-----------------
```

Out[20]: (Decimal('0.263'), Decimal('0.017'))

Aufgabe 2

In [21]:
```
g_sym, pk_sym, pf_sym, eta_sym, lam_sym, r_sym = sp.symbols('g, p_k, p_f, \\eta, v_\\text{lam}, r
params= [g_sym, pk_sym, pf_sym, eta_sym, lam_sym, r_sym]

v_lam = 2/9 * g_sym * (pk_sym - pf_sym) / (eta_sym * lam_sym) * r_sym**2 /1000
```

```python
r_val = df["r [mm]"].to_numpy()

theoretische_geschwindigkeiten = []
for n in range(len(lam_val)):

    values = np.array([[
        g_hd, err_g_hd,
        1385, 15,
        1148.2, 0.5,
        0.263, 0.017,
        float(lam_val[n]), float(lam_err[n]),
        r_val[n], 0.013
    ]])

    result = calc_everything(v_lam, params, values, [], False) # mm/s
    theoretische_geschwindigkeiten.append(result)
```

```
C:\Users\finnz\AppData\Local\Temp\ipykernel_5428\2988192940.py:16: FutureWarning: Calling float o
n a single element Series is deprecated and will raise a TypeError in the future. Use float(ser.i
loc[0]) instead
  float(lam_val[n]), float(lam_err[n]),
0.48392428102050733 \pm 0.05052235525249549
1.9093354900964767 \pm 0.17983905216023535
4.238285017843714 \pm 0.3906600926957551
7.434836681133249 \pm 0.6799769999384858
11.464935069037251 \pm 1.0447416235196376
16.29628409074963 \pm 1.4820389857941325
21.89823481492252 \pm 1.9890973782280608
28.241681781570904 \pm 2.5632830298524834
35.298967049145965 \pm 3.2020925464702312
```

In [22]:

```python
print(theoretische_geschwindigkeiten[4][0][1])

for n in range(len(lam_val)):
    try:
        print(round_sig_digs(theoretische_geschwindigkeiten[n][0][1], theoretische_geschwindigkei
    except IndexError:
        print("No")
```

```
1.0447416235196376
Aus den Messwert 0.48392428102050733 mit der Ungenauigkeit 0.05052235525249549 berechnet sich das
Ergebnis zu:
 0.48 \pm 0.05
-----------------

(Decimal('0.48'), Decimal('0.05'))
Aus den Messwert 1.9093354900964767 mit der Ungenauigkeit 0.17983905216023535 berechnet sich das
Ergebnis zu:
 1.91 \pm 0.18
-----------------

(Decimal('1.91'), Decimal('0.18'))
Aus den Messwert 4.238285017843714 mit der Ungenauigkeit 0.3906600926957551 berechnet sich das Er
gebnis zu:
 4.2 \pm 0.4
-----------------

(Decimal('4.2'), Decimal('0.4'))
Aus den Messwert 7.434836681133249 mit der Ungenauigkeit 0.6799769999384858 berechnet sich das Er
gebnis zu:
 7.4 \pm 0.7
-----------------

(Decimal('7.4'), Decimal('0.7'))
No
Aus den Messwert 16.29628409074963 mit der Ungenauigkeit 1.4820389857941325 berechnet sich das Er
gebnis zu:
 16 \pm 1.5
-----------------

('16', Decimal('1.5'))
Aus den Messwert 21.89823481492252 mit der Ungenauigkeit 1.9890973782280608 berechnet sich das Er
gebnis zu:
 21 \pm 1.10
-----------------

('21', Decimal('1.10'))
Aus den Messwert 28.241681781570904 mit der Ungenauigkeit 2.5632830298524834 berechnet sich das E
rgebnis zu:
 28 \pm 2.6
-----------------

('28', Decimal('2.6'))
Aus den Messwert 35.298967049145965 mit der Ungenauigkeit 3.2020925464702312 berechnet sich das E
rgebnis zu:
 35 \pm 3
-----------------

('35', Decimal('3'))
```

Reynolds

In [23]:
```python
rho_sym, v_sym, L_sym, eta_sym = sp.symbols('\\rho_f, v, L, \\eta')
params= [rho_sym, v_sym, L_sym, eta_sym]

re = rho_sym * v_sym * L_sym / eta_sym

# kg/m^3
rho_val = 1148.2
rho_err = 0.5
```

```python
# m
L_val = (df["d [mm]"]) /1000
L_err = (df["err d"]) /1000

print(L_val[0])

# kg / (m s)
eta_val = 0.263
eta_err = 0.017

# m/s

reynolds_zahlen = []
for n in range(len(lam_val)):
    values = np.array([[
        rho_val, rho_err,
        float(v_val[n]), float(v_err[n]),
        float(L_val[n]), float(L_err[n]),
        eta_val, eta_err,
    ]])

    result = calc_everything(re, params, values, [], False)
    reynolds_zahlen.append(result / 1000)

print(reynolds_zahlen)
```

```
0.001
2.761753205769616 \pm 0.22124673840248085
C:\Users\finnz\AppData\Local\Temp\ipykernel_5428\3383050494.py:26: FutureWarning: Calling float o
n a single element Series is deprecated and will raise a TypeError in the future. Use float(ser.i
loc[0]) instead
  float(v_val[n]), float(v_err[n]),
17.477099550362723 \pm 1.3626358459836931
59.53335637746285 \pm 4.139069944941397
130.58163911307904 \pm 8.8746131982553
253.8243876558493 \pm 17.807499574597585
393.31346555681165 \pm 27.258033673492573
627.5247694818122 \pm 45.42433146859816
884.2084997834144 \pm 66.93291396981454
1212.716518800169 \pm 97.2400921302415
[array([[0.00276175, 0.00022125]]), array([[0.0174771 , 0.00136264]]), array([[0.05953336, 0.0041
3907]]), array([[0.13058164, 0.00887461]]), array([[0.25382439, 0.0178075 ]]), array([[0.3933134
7, 0.02725803]]), array([[0.62752477, 0.04542433]]), array([[0.8842085 , 0.06693291]]), array
([[1.21271652, 0.09724009]])]
```

In [24]:

```python
for n in range(len(lam_val)):
    try:
        print(round_sig_digs(reynolds_zahlen[n][0][1], reynolds_zahlen[n][0][0]))
    except IndexError:
        print("No")
```

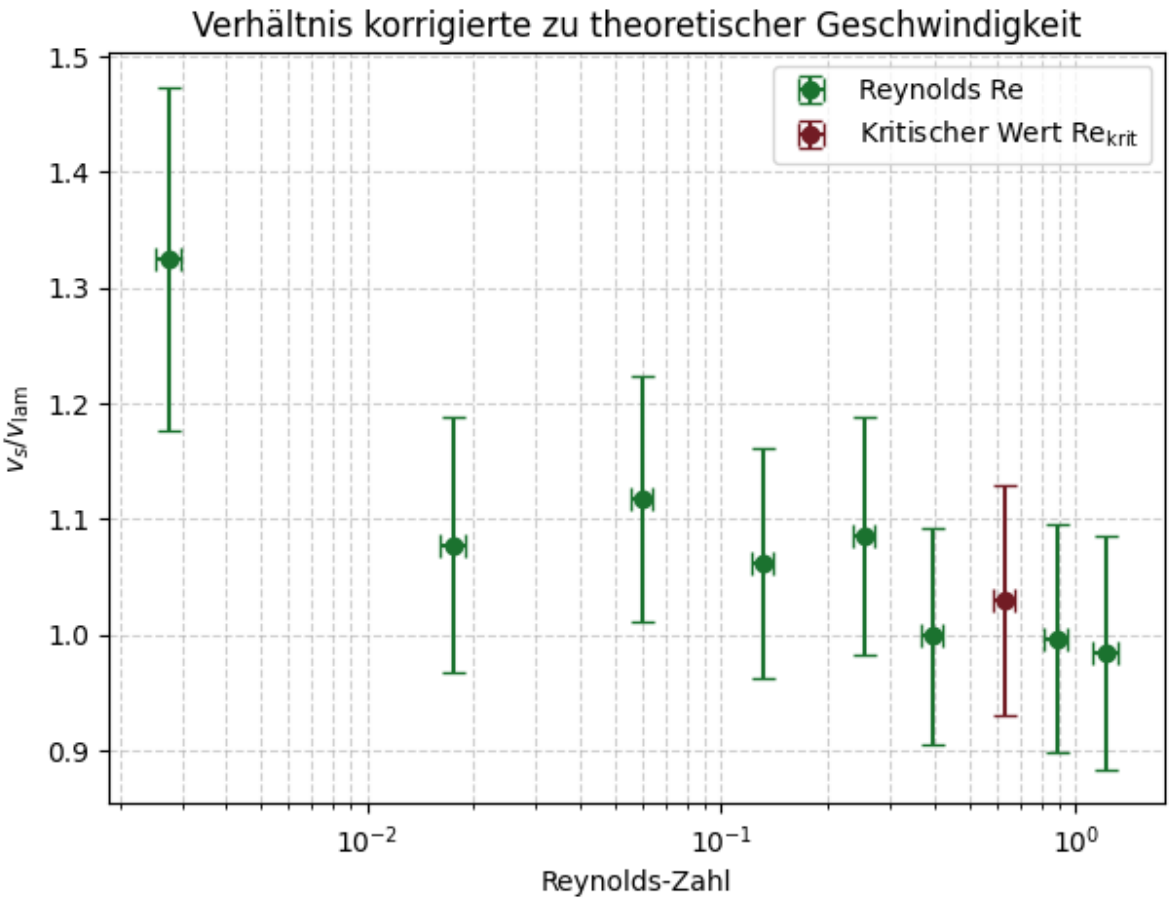
```
Aus den Messwert 0.0027617532057696162 mit der Ungenauigkeit 0.00022124673840248085 berechnet sic
h das Ergebnis zu:
 0.00276 \pm 0.00022
-----------------

(Decimal('0.00276'), Decimal('0.00022'))
Aus den Messwert 0.017477099550362724 mit der Ungenauigkeit 0.001362635845983693 berechnet sich d
as Ergebnis zu:
 0.0175 \pm 0.0014
-----------------

(Decimal('0.0175'), Decimal('0.0014'))
Aus den Messwert 0.05953335637746285 mit der Ungenauigkeit 0.004139069944941397 berechnet sich da
s Ergebnis zu:
 0.060 \pm 0.004
-----------------

(Decimal('0.060'), Decimal('0.004'))
Aus den Messwert 0.13058163911307905 mit der Ungenauigkeit 0.008874613198255301 berechnet sich da
s Ergebnis zu:
 0.131 \pm 0.009
-----------------

(Decimal('0.131'), Decimal('0.009'))
Aus den Messwert 0.2538243876558493 mit der Ungenauigkeit 0.017807499574597586 berechnet sich das
Ergebnis zu:
 0.254 \pm 0.018
-----------------

(Decimal('0.254'), Decimal('0.018'))
Aus den Messwert 0.3933134655568116 mit der Ungenauigkeit 0.02725803367349257 berechnet sich das
Ergebnis zu:
 0.393 \pm 0.027
-----------------

(Decimal('0.393'), Decimal('0.027'))
Aus den Messwert 0.6275247694818122 mit der Ungenauigkeit 0.04542433146859816 berechnet sich das
Ergebnis zu:
 0.63 \pm 0.05
-----------------

(Decimal('0.63'), Decimal('0.05'))
Aus den Messwert 0.8842084997834143 mit der Ungenauigkeit 0.06693291396981453 berechnet sich das
Ergebnis zu:
 0.88 \pm 0.07
-----------------

(Decimal('0.88'), Decimal('0.07'))
Aus den Messwert 1.2127165188001692 mit der Ungenauigkeit 0.0972400921302415 berechnet sich das E
rgebnis zu:
 1.2 \pm 0.10
-----------------

(Decimal('1.2'), Decimal('0.10'))
```

Lograitmischer Plot von v/v_lam über Re

In [25]:
```
# theoretische Geschwindigkeiten (Wert, Fehler)
v_lam_raw = [
np.array([[0.48392428, 0.05052236]]),
np.array([[1.90933549, 0.17983905]]),
```

```
np.array([[4.23828502, 0.39066009]]),
np.array([[7.43483668, 0.679977]]),
np.array([[11.46493507, 1.04474162]]),
np.array([[16.29628409, 1.48203899]]),
np.array([[21.89823481, 1.98909738]]),
np.array([[28.24168178, 2.56328303]]),
np.array([[35.29896705, 3.20209255]])
]

# Reynolds-Zahlen (Wert, Fehler)
re_raw = [
np.array([[0.00276175, 0.00022125]]),
np.array([[0.0174771, 0.00136264]]),
np.array([[0.05953336, 0.00413907]]),
np.array([[0.13058164, 0.00887461]]),
np.array([[0.25382439, 0.0178075]]),
np.array([[0.39331347, 0.02725803]]),
np.array([[0.62752477, 0.04542433]]),
np.array([[0.8842085, 0.06693291]]),
np.array([[1.21271652, 0.09724009]])
]

# Messwerte
vs_val = np.array([
0.641447, 2.057646, 4.736364, 7.896311, 12.44186,
16.276276, 22.546201, 28.151899, 34.753086
])

vs_err = np.array([
0.025777, 0.085947, 0.114695, 0.158296, 0.333627,
0.40123, 0.730266, 1.105647, 1.646186
])


v_lam = np.array([v[0,0] for v in v_lam_raw])
v_lam_err = np.array([v[0,1] for v in v_lam_raw])

re = np.array([r[0,0] for r in re_raw])
re_err = np.array([r[0,1] for r in re_raw])


# Verhältnis
ratio = vs_val / v_lam

# Fehlerfortpflanzung
ratio_err = ratio * np.sqrt(
    (vs_err / vs_val)**2 +
    (v_lam_err / v_lam)**2
)


plt.figure(figsize=(7,5))

# Kritischer Knick
i = 6

mask = np.arange(len(re)) != i

plt.errorbar(
    re[mask], ratio[mask],
    xerr=re_err[mask],
    yerr=ratio_err[mask],
```

```
    fmt='o',
    capsize=4,
    color=gruen,
    label="Reynolds Re"
)

# kritischer Wert
plt.errorbar(
    re[i], ratio[i],
    xerr=re_err[i],
    yerr=ratio_err[i],
    fmt='o',
    capsize=4,
    color=weinrot,
    label=r"Kritischer Wert Re$_\text{krit}$"
)

plt.xscale("log")

plt.xlabel("Reynolds-Zahl")
plt.ylabel(r"$v_s / v_\text{lam}$")
plt.title("Verhältnis korrigierte zu theoretischer Geschwindigkeit")
plt.grid(True, which="both", linestyle="--", alpha=0.6)

plt.legend()
plt.savefig("../img/plot/ver_ges_gegen_re.pdf", dpi=300)
plt.show()
```



Aufgabe 3

In [56]:
```
# cm^3
vol = np.array([5, 10, 15, 20, 25, 30]) /1e6
```

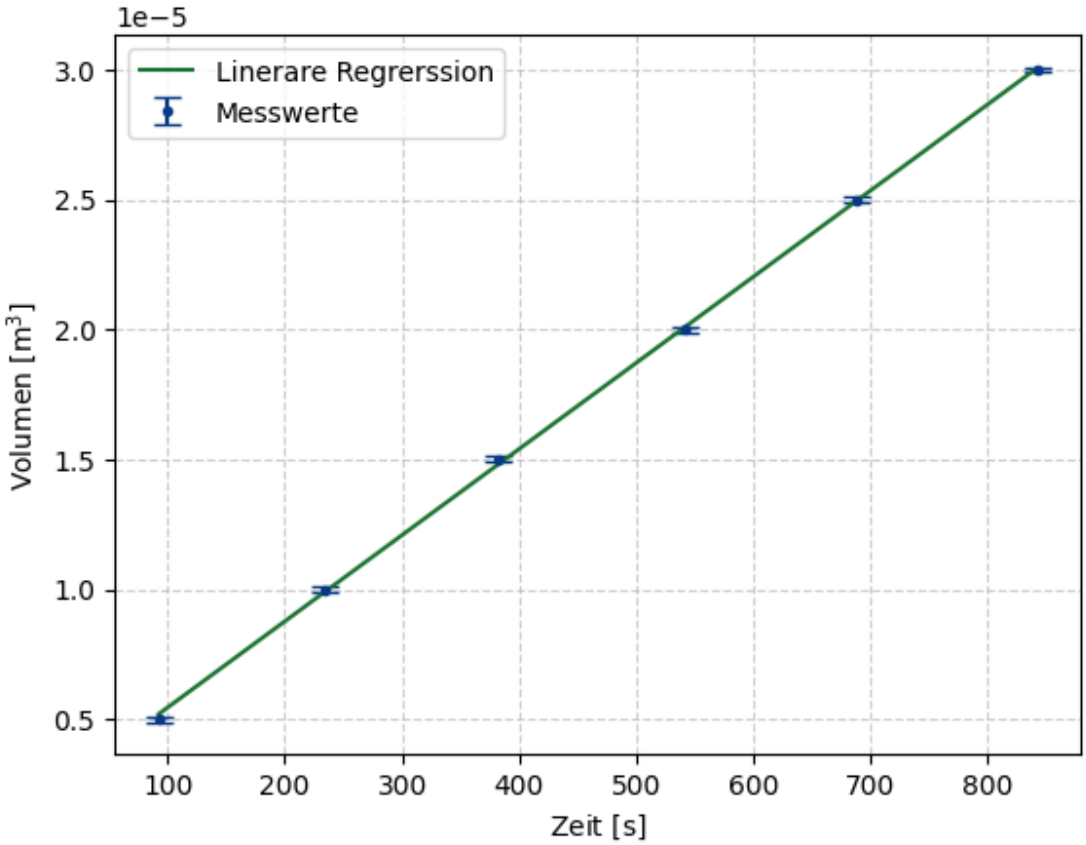
```
vol_err = 0.10 /1e6

# s
t = np.array([92.4, 233.8, 382.8, 540.9, 688.4, 841.9])
t_err = 1.0

# mm
h_a = 531 / 1000
h_e = 524 / 1000
h_err = 3 / 1000

# mm
d_kap = 1.5  / 1000
r_kap = d_kap / 2  / 1000
d_kap_err = 0.01  / 1000
r_kap_err = d_kap_err / 2  / 1000

l_kap = 100.0  / 1000
l_kap_err = 0.5  / 1000
```

Volumenfluss bestimmen:

In [61]:
```
y = vol
y_err = y * np.sqrt((vol_err/vol)**2 + (t_err/t)**2)

# lineare regression
def linear(x, a, b):
    return a*x + b

popt, pcov = curve_fit(linear, t, y, sigma=y_err, absolute_sigma=True)

slope = popt[0]
intercept = popt[1]

slope_err = np.sqrt(pcov[0,0])
intercept_err = np.sqrt(pcov[1,1])

# smooth fit line
t_fit = np.linspace(min(t), max(t), 200)
y_fit = linear(t_fit, slope, intercept)

# plot
plt.errorbar(
    t,
    y,
    yerr=y_err,
    fmt='.',
    label="Messwerte",
    capsize=5,
    color=blau)


plt.plot(
    t_fit,
    y_fit,
    label="Linerare Regrerssion",
    color= gruen)

plt.grid(True, which="both", linestyle="--", alpha=0.6)
plt.xlabel("Zeit [$\mathrm{s}$]")
plt.ylabel(r'Volumen [$\mathrm{m^3}$]')
```

```
plt.legend()
plt.savefig("../img/plot/volumenfluss.pdf", dpi=300)
plt.show()

# print results
print(f"Steigung = {slope:.6e} \\pm {slope_err:.6e}")
print(f"y-Achsenabschnitt = {intercept:.6e} \\pm {intercept_err:.6e}")
```

```
<>:38: SyntaxWarning: invalid escape sequence '\m'
<>:38: SyntaxWarning: invalid escape sequence '\m'
C:\Users\finnz\AppData\Local\Temp\ipykernel_5428\2296362686.py:38: SyntaxWarning: invalid escape
sequence '\m'
  plt.xlabel("Zeit [$\mathrm{s}$]")
```




```
Steigung = 3.318474e-08 \pm 1.732220e-10
y-Achsenabschnitt = 2.125926e-06 \pm 9.313616e-08
```

In [58]:

```
rhof_sym, g_sym, ha_sym, he_sym = sp.symbols(r'\rho_f, g, h_\text{A}, h_\text{E}')
params = [rhof_sym, g_sym, ha_sym, he_sym]

p_diff = rhof_sym * g_sym / 2 * (ha_sym + he_sym)

pf_val = 1148.2
pf_err = 0.5

values = np.array([
    [
    pf_val, pf_err,
    g_hd, err_g_hd,
    h_a, h_err,
    h_e, h_err
    ]
])

print(values)
```

```
calc_everything(p_diff, params, values, [], False)
```

```
[[1.14820e+03 5.00000e-01 9.80984e+00 2.00000e-05 5.31000e-01 3.00000e-03
  5.24000e-01 3.00000e-03]]
5941.5797469200015 \pm 24.033507725013767
```

Out[58]: array([[5941.57974692, 24.03350773]])

Eta bestimmen

In [59]:
```
p_sym, R_sym, L_sym, Vt_sym = sp.symbols('p, R, L, \\dot{V}')
params = [p_sym, R_sym, L_sym, Vt_sym]

# Pa
rho_val = 594
rho_err = 24

# m^3/s * 10^{-8}
vt_val = 3.318
vt_err = 0.017

values = np.array([
    [
    rho_val, rho_err,
    r_kap, r_kap_err,
    l_kap, l_kap_err,
    vt_val, vt_err,
    ]
])

eta = (sp.pi * p_sym * R_sym**4) / (8 * L_sym * Vt_sym)

calc_everything(eta, params, values, [], True)
```

```
2.224410838707949e-22 \pm 1.0885641540387825e-23
gegebene Funktion:
Die gegebene Funktion lautet:
```

$$\frac{\pi R^4 p}{8L\dot{V}}$$

```
Hier ist der dazugehörige Latex code:
```

```
\frac{\pi R^{4} p}{8 L \dot{V}}
```

Kopieren

```
Formel des absoluten Fehlers der gegebenen Funktion:
Die gegebene Funktion lautet:
```

$$\frac{\pi}{8}\sqrt{\frac{R^6}{L^4\dot{V}^4}\left(L^2R^2p^2\Delta\dot{V}^2 + L^2\dot{V}^2\left(R^2\Delta p^2 + 16p^2\Delta R^2\right) + R^2\dot{V}^2p^2\Delta L^2\right)}$$

```
Hier ist der dazugehörige Latex code:
```

```
\frac{\pi}{8} \sqrt{\frac{R^{6}}{L^{4} \dot{V}^{4}} \left(L^{2} R^{2} p^{2} \Delta \dot{V}^{2} + L^{2} \dot{V}^{2} \left(R^{2} \Delta p^{2} + 16 p^{2} \Delta R^{2}\right) + R^{2} \dot{V}^{2} p^{2} \Delta L^{2}\right)}
```

Kopieren

Formel des relativen Fehlers der gegebenen Funktion:
Die gegebene Funktion lautet:

$$\sqrt{\frac{\Delta p^{2}}{p^{2}} + \frac{\Delta \dot{V}^{2}}{\dot{V}^{2}} + \frac{16 \Delta R^{2}}{R^{2}} + \frac{\Delta L^{2}}{L^{2}}}$$

Hier ist der dazugehörige Latex code:

```
\sqrt{\frac{\Delta p^{2}}{p^{2}} + \frac{\Delta \dot{V}^{2}}{\dot{V}^{2}} + \frac{16 \Delta R^{2}}{R^{2}} + \frac{\Delta L^{2}}{L^{2}}}
```

Kopieren

Out[59]: array([[2.22441084e-22, 1.08856415e-23]])

Reynolds-Zahl

In [60]:
```python
Vt, rho_f, L, R = sp.symbols(r'\dot{V}, \rho_f, L, R')
params = [Vt, rho_f, L, R]

values = np.array([
    [
    vt_val, vt_err,
    pf_val, pf_err,
    l_kap, l_kap_err,
    d_kap, d_kap_err,
    ]
])

Re_hp = 2 * (Vt * rho_f) / (L * R * sp.pi)

calc_everything(Re_hp, params, values, [])
```

16168986.116629949 \pm 158328.16746938726
gegebene Funktion:
Die gegebene Funktion lautet:

$$\frac{2 \dot{V} \rho_{f}}{\pi L R}$$

Hier ist der dazugehörige Latex code:

```
\frac{2 \dot{V} \rho_{f}}{\pi L R}
```

Kopieren

Formel des absoluten Fehlers der gegebenen Funktion:
Die gegebene Funktion lautet:

$$\frac{2}{\pi} \sqrt{\frac{1}{L^4 R^4} \left(L^2 R^2 \left(\dot{V}^2 \Delta \rho_f^2 + \rho_f^2 \Delta \dot{V}^2\right) + L^2 \dot{V}^2 \rho_f^2 \Delta R^2 + R^2 \dot{V}^2 \rho_f^2 \Delta L^2\right)}$$

Hier ist der dazugehörige Latex code:

```
\frac{2}{\pi} \sqrt{\frac{1}{L^{4} R^{4}} \left(L^{2} R^{2} \left(\dot{V}^{2} \Delta \rho_{f}^{2} + \rho_{f}^{2} \Delta \dot{V}^{2}\right) + L^{2} \dot{V}^{2} \rho_{f}^{2} \Delta R^{2} + R^{2} \dot{V}^{2} \rho_{f}^{2} \Delta L^{2}\right)}
```

Kopieren

Formel des relativen Fehlers der gegebenen Funktion:
Die gegebene Funktion lautet:

$$\sqrt{\frac{\Delta \rho_f^2}{\rho_f^2} + \frac{\Delta \dot{V}^2}{\dot{V}^2} + \frac{\Delta R^2}{R^2} + \frac{\Delta L^2}{L^2}}$$

Hier ist der dazugehörige Latex code:

```
\sqrt{\frac{\Delta \rho_{f}^{2}}{\rho_{f}^{2}} + \frac{\Delta \dot{V}^{2}}{\dot{V}^{2}} + \frac{\Delta R^{2}}{R^{2}} + \frac{\Delta L^{2}}{L^{2}}}
```

Kopieren

Out[60]: array([[16168986.11662995, 158328.16746939]])

Signifikante Abweichungen der Geschwindigkeiten

In [ ]:

```python
# v_gem
v_gem = np.array([0.63, 2.00, 4.55, 7.48, 11.63, 15.02, 20.53, 25.3, 30.9])
v_gem_err = np.array([0.03, 0.08, 0.07, 0.07, 0.23, 0.20, 0.18, 0.4, 0.3])

# v_korr
v_korr = np.array([0.641, 2.06, 4.74, 7.90, 12.4, 16.3, 22.5, 28.15, 34.8])
v_korr_err = np.array([0.026, 0.09, 0.11, 0.16, 0.3, 0.4, 0.7, 1.11, 1.6])

# v_theo
v_theo = np.array([0.48, 1.91, 4.2, 7.4, 11.5, 16.3, 21.90, 28.2, 35])
v_theo_err = np.array([0.05, 0.18, 0.4, 0.7, 1.0, 1.5, 1.10, 2.6, 3])

def sigma_abweichung(p1, p2, err_p1=0, err_p2=0):
    if err_p1 == err_p2 == 0:
        raise ValueError("Für die Sigma-Abweichung muss mindestens ein Wert fehlerbehaftet sein!"
    abweichung = abs(p1 - p2) / np.sqrt(err_p1**2 + err_p2**2)
    return abweichung

sigma_list = []

for vg, vg_err, vk, vk_err in zip(v_gem, v_gem_err, v_korr, v_korr_err):
    sigma = sigma_abweichung(vg, vk, vg_err, vk_err)
    sigma_list.append(f"{sigma:.2f}")  # rundet auf 2 Nachkommastellen und hängt \sigma an

# In ein numpy array umwandeln, wenn nötig
sigma_array = np.array(sigma_list)

print(sigma_array)
```

```
['0.28' '0.50' '1.46' '2.40' '2.04' '2.86' '2.73' '2.42' '2.40']
```

In [104…

```python
sigma_list = []

for vt, vt_err, vk, vk_err in zip(v_theo, v_theo_err, v_korr, v_korr_err):
    sigma = sigma_abweichung(vt, vk, vt_err, vk_err)
    sigma_list.append(f"{sigma:.2f}")  # rundet auf 2 Nachkommastellen und hängt \sigma an

# In ein numpy array umwandeln, wenn nötig
sigma_array = np.array(sigma_list)

print(sigma_array)
```

```
['2.86' '0.75' '1.30' '0.70' '0.86' '0.00' '0.46' '0.02' '0.06']
```

Abbildungsverzeichnis

1.I Gedankliche Darstellung der Viskosität. [Wik26] 3
1.II Schematische Darstellung der Strömungsregime um eine Kugel: Laminar (links) mit symmetrischem Stromlinienmuster, (rechts) mit unregelmäßigen Wirbeln. 4
1.III Schematische Darstellung für den zweiten Versuchsteil: Eine Flüssigkeit fließt durch ein Rohr, wobei die Geschwindigkeitsverteilung parabolisch ist. 5

3.I Dartsellung des Versuchaufbaus. Orange: Wichtig für beide Aufgabenteile. Blau: Wichtig zur Bestimmung nach Stokes. Magenta: Wichtig zur Bestimmung nach Hagen-Poiseuille . 9
3.II Dartsellung des Versuchaufbaus. Orange: Wichtig für beide Aufgabenteile. Blau: Wichtig zur Bestimmung nach Stokes. Magenta: Wichtig zur Bestimmung nach Hagen-Poiseuille . 10

4.I Plot der Geschwindigkeiten gegen den quadrierten Kugelradius. Die Lineare Regression ist durch die ersten fünf Messwerte der korrigierten Werte gelegt. Die lineare Nährung zeigt sich damit als nicht gültig. 13
4.II Graph der Dichte von PEG in Temperaturabhänigkeit. Eingezeichnet sind die Werte für die Temperatur $T_1 = (21.1 \pm 0.5)°\mathrm{C}$ und die für $T_2 = (21.5 \pm 1.0)°\mathrm{C}$. Die Werte sind noch nicht gerundet. Bestimmt wurden die Werte über das Grafikprogramm „Inkscape“. 15
4.III Keine eindeutigen Messergebnisse. Der kritische Punkt wird nur erahnt. 16
4.IV Plot der Messwerte (blau) mit linerarer Regression (grün). 17

Tabellenverzeichnis

2..1 Sinkgeschwindigkeit nach Kugeldurchmesser 8
2..2 Volumenfluss des PEG 8

4.I Wichtigen Messwerte aus dem Messprotokoll zusammengefasst. 13
4.II Theoretische Sinkgeschwindigkeit v_{lam} und Reynoldszahl in Abhänigkeit von r, mit der Ungenauigkeit $\Delta r = 0.013\text{m} \cdot 10^{-3}$. 16

5.I Vergleich und Bestimmugn der Signifikanz der verschiedenen Geschwindigkeiten für die Kugel 20

Literaturverzeichnis

[Dud26] Duden. Duden rechtschreibprüfer, 2026.

[Fin26] Finn Zeumer. Pap 2, 2026. Zugriff am 17. Februar 2026.

[Wag25] Dr. J. Wagner. *Physikalisches Praktikum PAP 1 für Studierende der Physik*, pages 4–28. Universität Heidelberg, 2025.

[Wag26] Dr. J. Wagner. *Physikalisches Anfängerpraktikum PAP 2.1 für Studierende der Physik*, chapter 212. Universität Heidelberg, 2026.

[Wik26] Wikipedia. Viskosität, 2026. Zugriff am 17. Februar 2026.